

Lecture: 09

Token Based Distributed Mutual Exclusion Algorithms



Rajiv Misra

Dept. of Computer Science & Engineering
Indian Institute of Technology Patna
rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have discussed about quorum based approaches i.e. **Maekawa's Algorithm and Agarwal-El Abbadi Quorum-Based Algorithm.**

Content of this Lecture:

- In this lecture, we will discuss about token based approaches i.e. **Suzuki-Kasami's Broadcast Algorithm and Raymond's Tree-Based Algorithm .**

(iii) Token Based Approaches

Introduction

- In token-based algorithms, a **unique token is shared** among the sites. A site is allowed to enter its CS if it possesses the token.
- Token-based algorithms **use sequence numbers** instead of timestamps. (Used to distinguish between old and current requests.)
- The correctness proof of token-based algorithms, that they enforce mutual exclusion, is trivial because an **algorithm guarantees mutual exclusion so long as a site holds the token** during the execution of the CS.
- In this lecture, we will discuss about two token based approaches i.e. (i) Suzuki-Kasami's Broadcast Algorithm and (ii) Raymond's Tree-Based Algorithm .

(i) Suzuki-Kasami's Broadcast Algorithm

- In Suzuki–Kasami's algorithm, if a site wants to enter the CS and it does not have the token, it broadcasts a **REQUEST** message for the token to all other sites.
- A site which possesses the token sends it to the requesting site upon the receipt of its **REQUEST** message.
- If a site receives a **REQUEST** message when it is executing the CS, it sends the token only after it has completed the execution of the CS.

Contd...

This algorithm must efficiently address the following two design issues:

(1) How to distinguish an outdated REQUEST message from a current REQUEST message:

- Due to variable message delays, a site may receive a token request message after the corresponding request has been satisfied.
- If a site can not determine if the request corresponding to a token request has been satisfied, it may dispatch the token to a site that does not need it.
- This will not violate the correctness, however, this may seriously degrade the performance.

(2) How to determine which site has an outstanding request for the CS:

- After a site has finished the execution of the CS, it must determine what sites have an outstanding request for the CS so that the token can be dispatched to one of them.

Contd...

The first issue is addressed in the following manner:

- A **REQUEST** message of site S_j has the form **REQUEST(j, n)** where n ($n=1, 2, \dots$) is a sequence number which indicates that site S_j is requesting its n^{th} CS execution.
- A site S_i keeps an array of integers $RN_i[1..N]$ where $RN_i[j]$ denotes the largest sequence number received in a **REQUEST** message so far from site S_j .
- When site S_i receives a **REQUEST(j, n)** message, it sets $RN_i[j] := \max(RN_i[j], n)$.
- When a site S_i receives a **REQUEST(j, n)** message, the request is outdated if $RN_i[j] > n$.

Contd...

The second issue is addressed in the following manner:

- The token consists of a **queue of requesting sites, Q** , and **an array of integers $LN[1..N]$** , where **$LN[j]$ is the sequence number of the request which site S_j executed most recently.**
- After executing its CS, a site S_i updates $LN[i] := RN_i[i]$ to indicate that its request corresponding to sequence number $RN_i[i]$ has been executed.
- At site S_i **if $RN_i[j] = LN[j] + 1$** , then site S_j is currently requesting token.

The Algorithm

Requesting the critical section

- (a) If requesting site S_i does not have the token, then it increments its sequence number, $RN_i[i]$, and sends a **REQUEST(i, sn)** message to all other sites. (' sn ' is the updated value of $RN_i[i]$.)
- (b) When a site S_j receives this message, it sets $RN_j[i]$ to $\max(RN_j[i], sn)$. If S_j has the idle token, then it sends the token to S_i if $RN_j[i] = LN[i] + 1$.

Executing the critical section

- (c) Site S_i executes the CS after it has received the token.

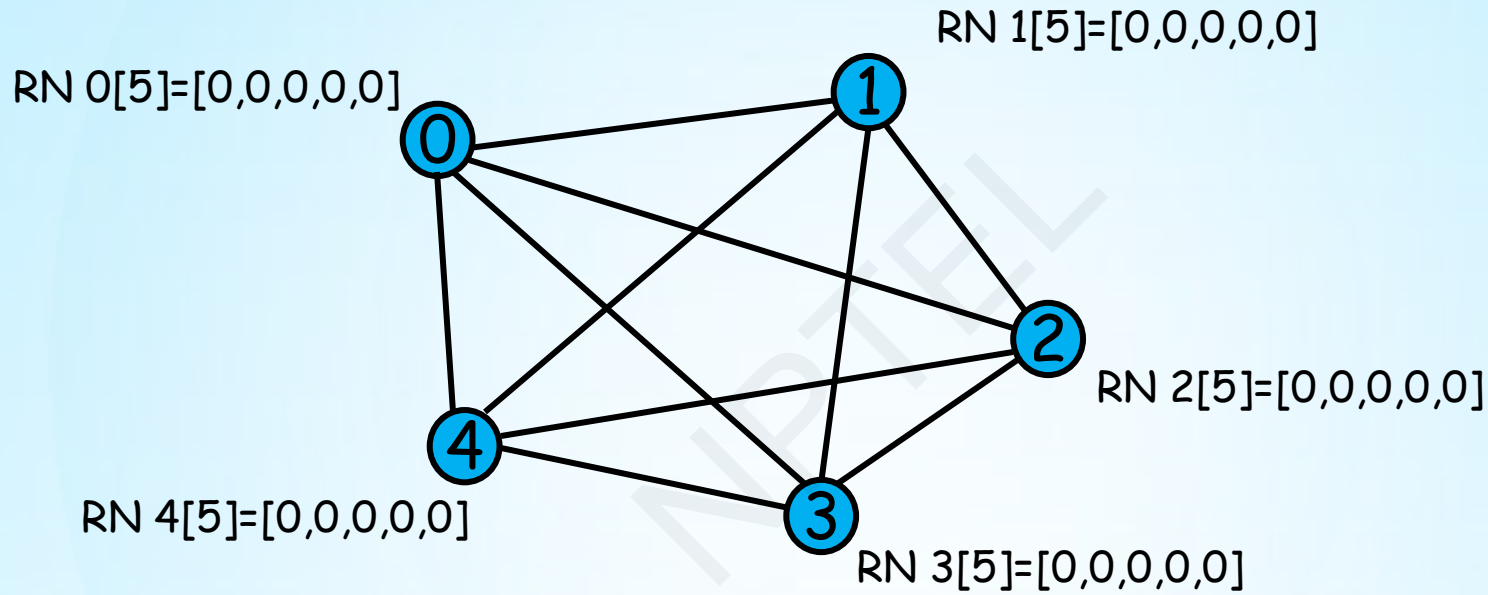
The Algorithm

Releasing the critical section

Having finished the execution of the CS, site S_i takes the following actions:

- (d) It sets $LN[i]$ element of the token array equal to $RN_i[i]$.
- (e) For every site S_j whose id is not in the token queue, it appends its id to the token queue if $RN_i[j] = LN[j] + 1$.
- (f) If the token queue is nonempty after the above update, S_i deletes the top site id from the token queue and sends the token to the site indicated by the id.

Suzuki-Kasami's Broadcast Algorithm: Example

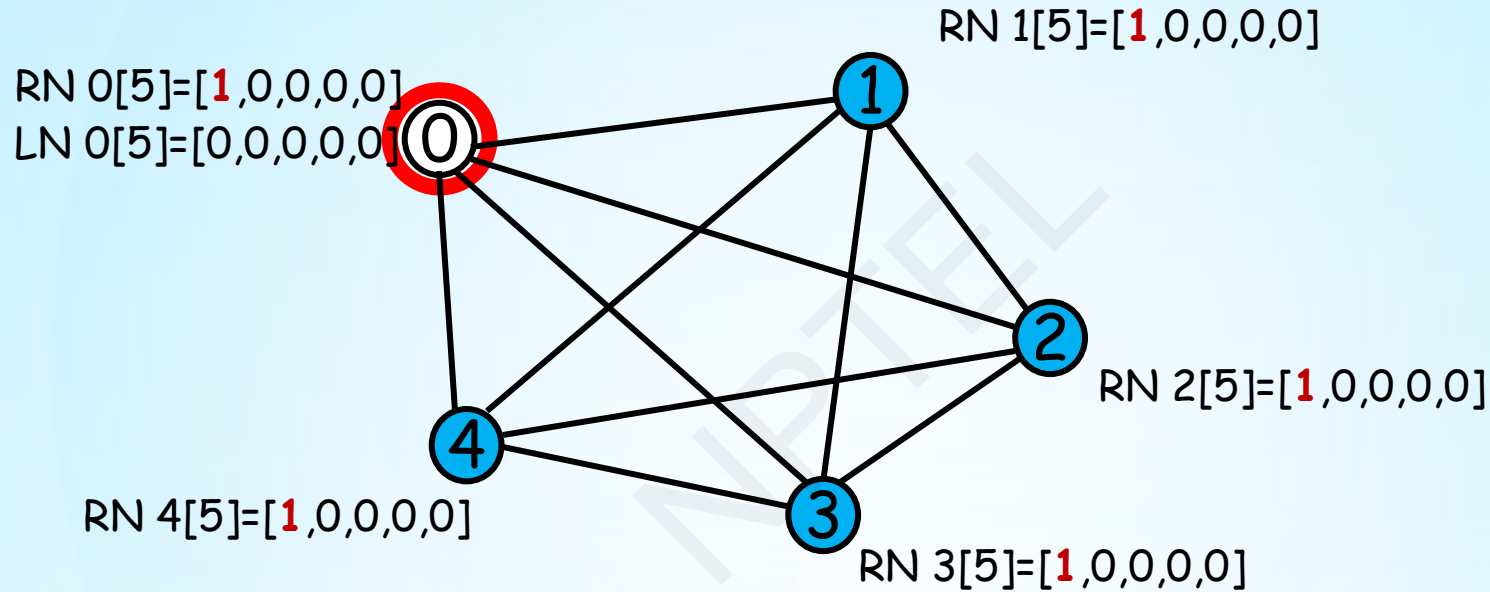


There are 5 sites initially and non are requesting

$RN[j]$ denotes the sequence no of the *latest request* from process j

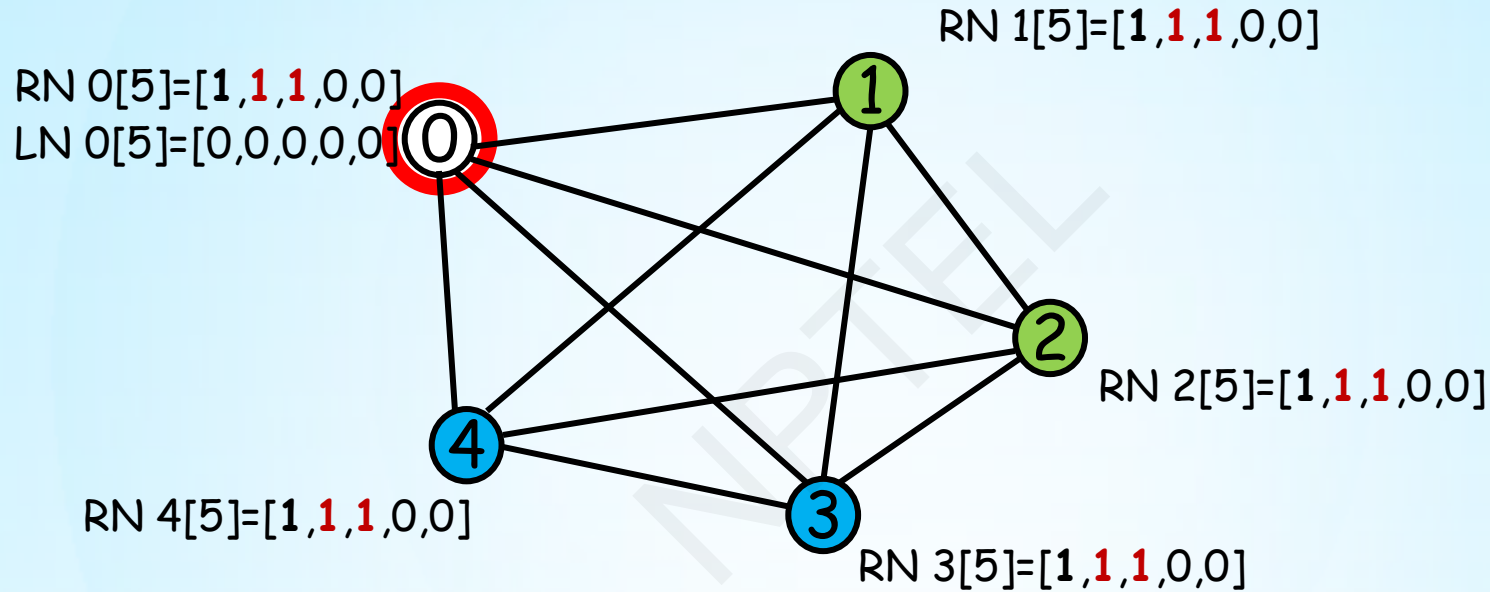
$LN[j]$ denotes the sequence number of *the latest visit to CS* for process j .

Suzuki-Kasami's Broadcast Algorithm: Example



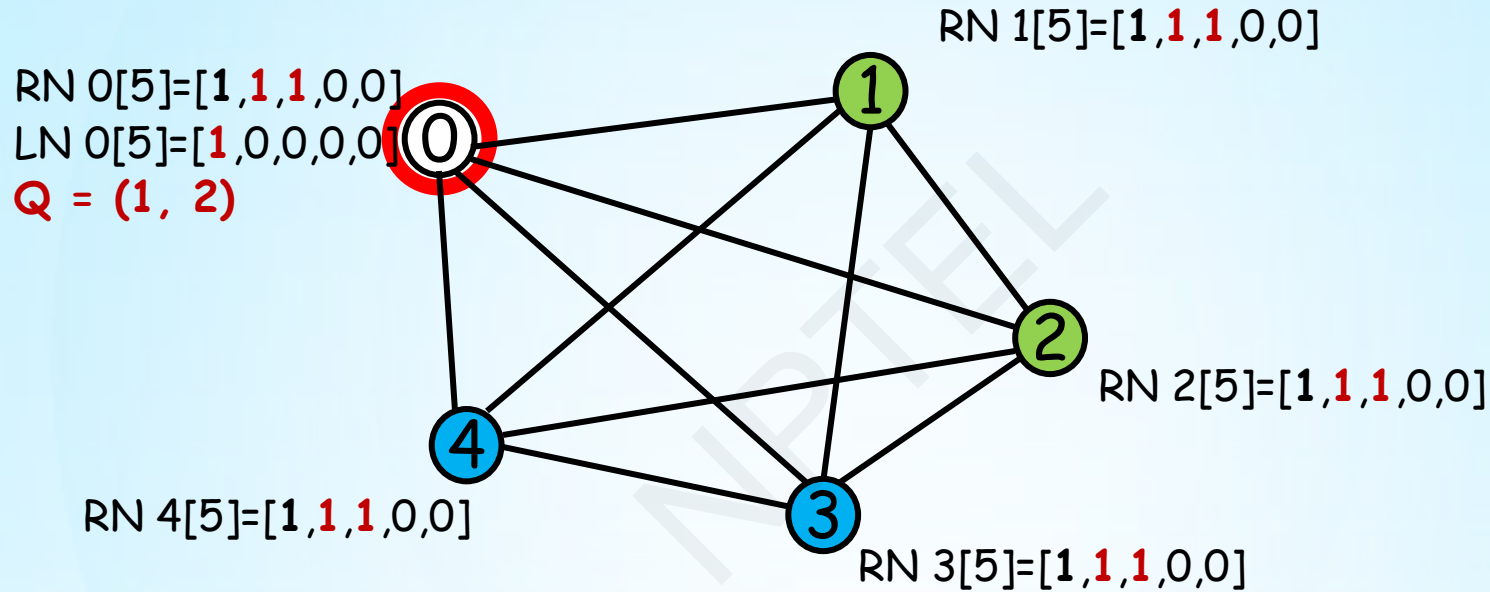
Initial state: Site 0 has sent a request to all, and grabbed the token

Suzuki-Kasami's Broadcast Algorithm: Example



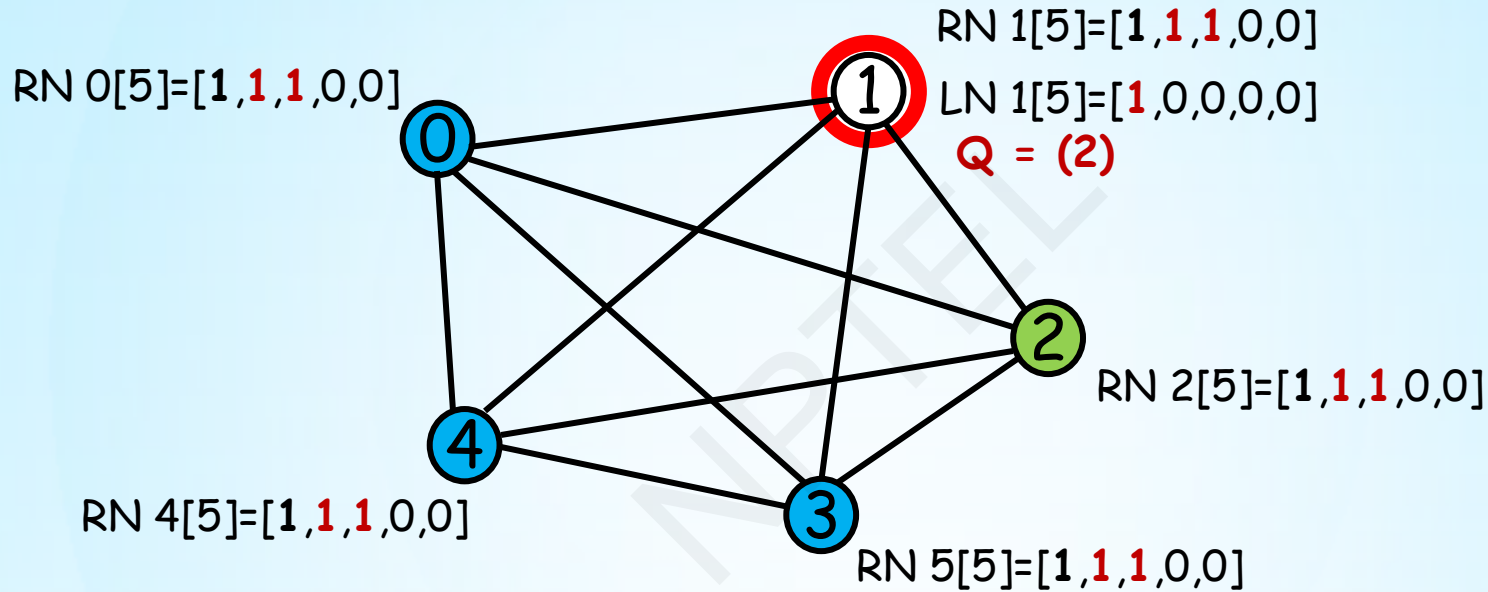
Site 1 & 2 send requests to enter CS

Suzuki-Kasami's Broadcast Algorithm: Example



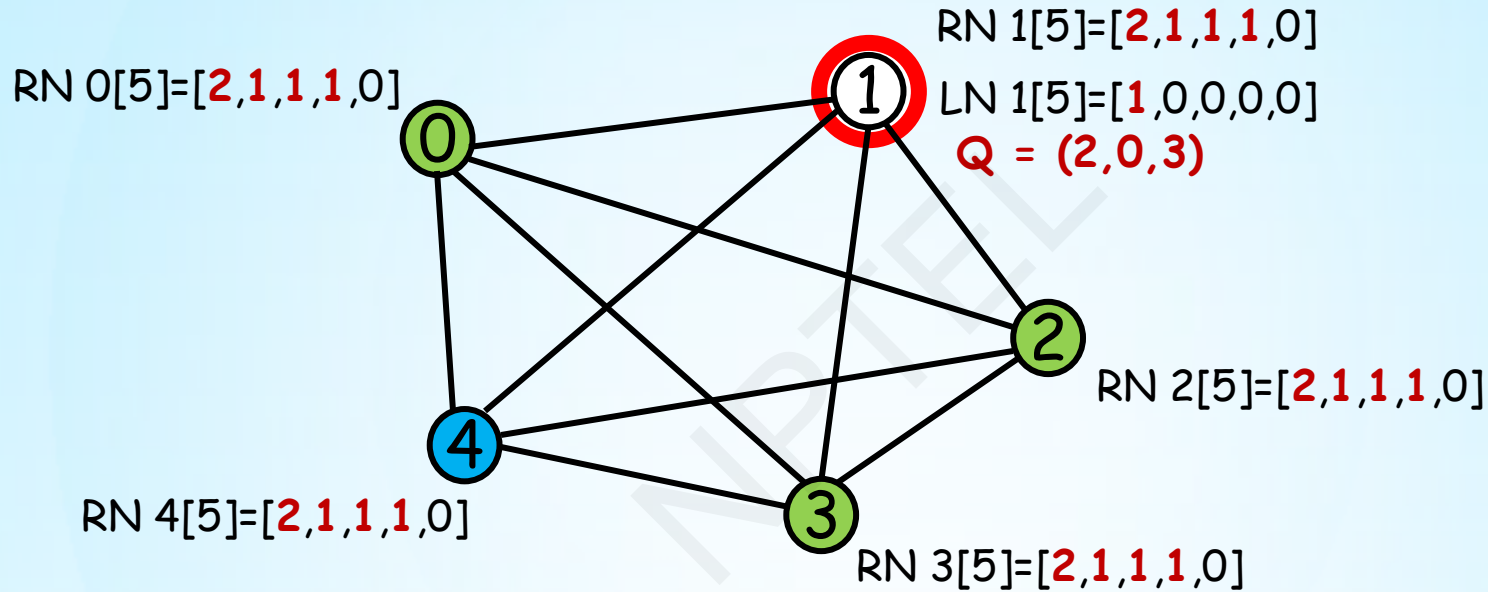
Site 0 prepares to exit CS

Suzuki-Kasami's Broadcast Algorithm: Example



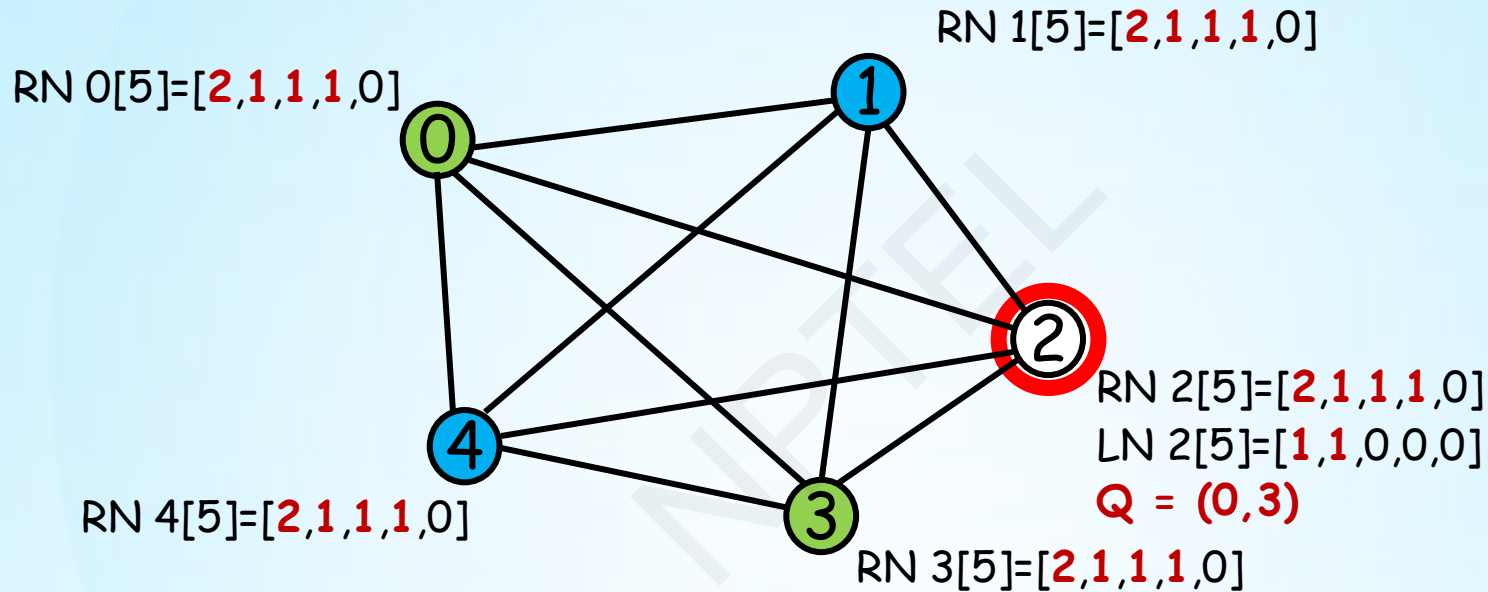
Site 0 passes token (Q and LN) to Site 1

Suzuki-Kasami's Broadcast Algorithm: Example



Site 0 and Site 3 send requests

Suzuki-Kasami's Broadcast Algorithm: Example



Site 1 sends token to Site 2

Correctness

Mutual exclusion is guaranteed because there is only one token in the system and a site holds the token during the CS execution.

Theorem: A requesting site enters the CS in finite time.

Proof:

- Token request messages of a site S_i reach other sites in finite time.
- Since one of these sites will have token in finite time, site S_i 's request will be placed in the token queue in finite time.
- Since there can be **at most $N - 1$ requests** in front of this request in the token queue, site S_i will get the token and execute the CS in finite time.

Performance

- No message is needed and the **synchronization delay is zero if a site holds the idle token** at the time of its request.
- If a site does not hold the token when it makes a request, the algorithm requires **N messages to obtain the token.**
- **Synchronization delay** in this algorithm is **0 or T .**

(ii) Raymond's Tree-Based Algorithm

- This algorithm uses a **spanning tree** to reduce the number of messages exchanged per critical section execution.
- The network is viewed as a graph, a spanning tree of a network is a **tree that contains all the N nodes**.
- The algorithm assumes that the underlying network **guarantees message delivery**. All nodes of the network are '**completely reliable**'.

Contd...

- The algorithm operates on a **minimal spanning tree** of the network topology or a logical structure imposed on the network.
- The algorithm assumes the network nodes to be arranged in an **unrooted tree structure**.
- Figure 9.1 shows a spanning tree of seven nodes A, B, C, D, E, F, and G.
- Messages between nodes traverse along the **undirected edges** of the tree.

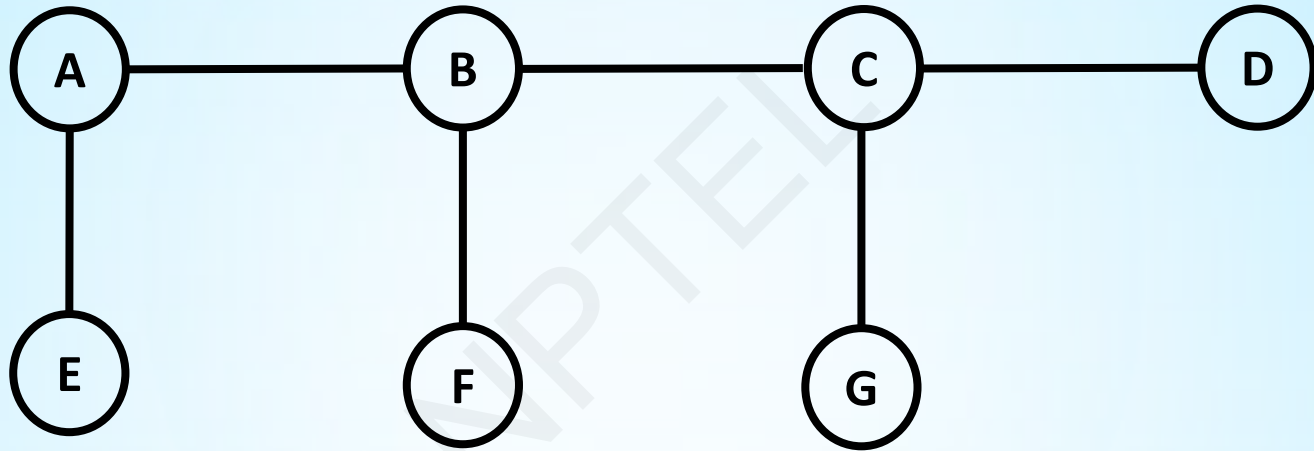


Figure 9.1: A tree of 15 sites

Contd...

- A node needs to hold information about and communicate only to its **immediate-neighboring nodes**.
- Similar to the concept of tokens used in token-based algorithms, this algorithm uses a concept of privilege.
- Only one node can be in possession of the privilege (called the **privileged node**) at any time, except when the privilege is in transit from one node to another in the form of a **PRIVILEGE message**.
- When there are no nodes requesting for the privilege, it remains in possession of the node that last used it.

The HOLDER Variables

- Each node maintains a **HOLDER** variable that provides information about the placement of the privilege in relation to the node itself.
- A node stores in its **HOLDER** variable the identity of a node that it thinks has the privilege or leads to the node having the privilege.
- For two nodes X and Y, if **HOLDER_x = Y**, we could redraw the undirected edge between the nodes X and Y as a directed edge from X to Y.
- For instance, if node G holds the privilege, Figure 9.2 can be redrawn with logically directed edges as shown in the Figure 9.3.

The HOLDER Variables

- The **shaded node in Figure 9.2 represents the privileged node.**
- The following will be the values of the **HOLDER** variables of various nodes:

$\text{HOLDER}_A = B$

$\text{HOLDER}_B = C$

$\text{HOLDER}_C = G$

$\text{HOLDER}_D = C$

$\text{HOLDER}_E = A$

$\text{HOLDER}_F = B$

$\text{HOLDER}_G = \text{self}$

Raymond's Algorithm: Example (1)

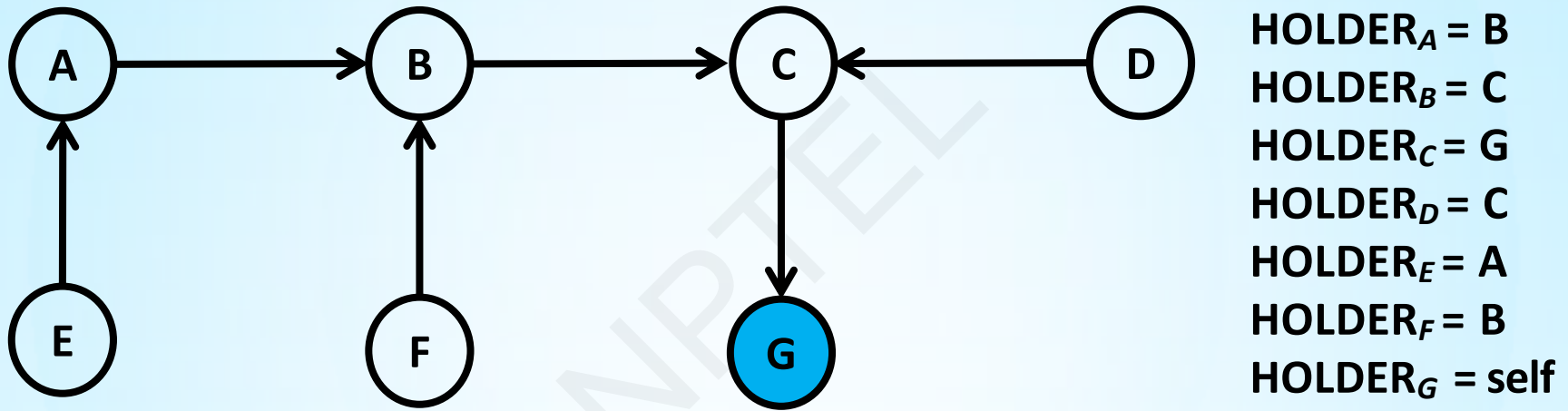


Figure 9.2: Tree with logically directed edges, all pointing in a direction towards node G - the privileged node.

Raymond's Algorithm: Example (1)

- Now suppose node B that does not hold the privilege wants to execute the critical section.
- B sends a **REQUEST** message to **HOLDER_B**, i.e., C, which in turn forwards the **REQUEST** message to **HOLDER_C**, i.e., G.
- The privileged node G, if it no longer needs the privilege, sends the **PRIVILEGE** message to its neighbor C, which made a request for the privilege, and resets **HOLDER_G** to C.
- Node C, in turn, forwards the **PRIVILEGE** to node B, since it had requested the privilege on behalf of B. Node C also resets **HOLDER_C** to B.
- The tree in Figure 9.2 will now look as in Figure 9.3.

Raymond's Algorithm: Example (1)

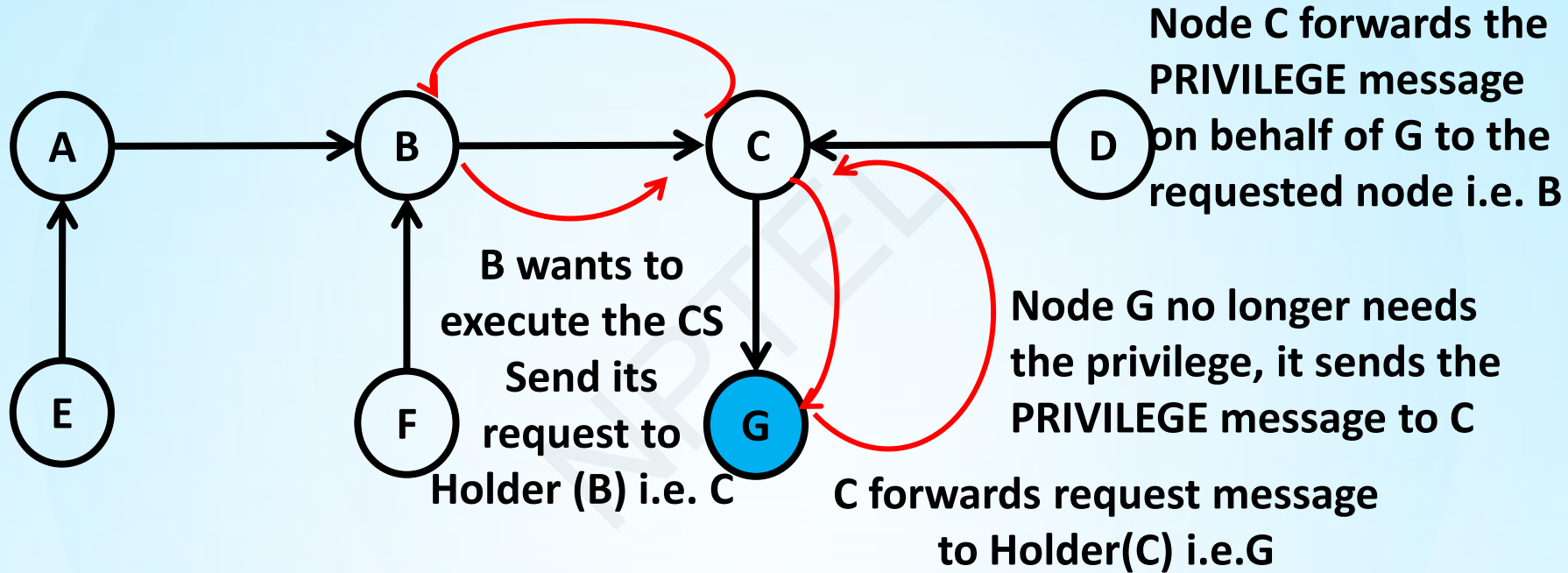


Figure 9.3: Tree with logically directed edges, all pointing in a direction towards node G - the privileged node.

Raymond's Algorithm: Example (1)

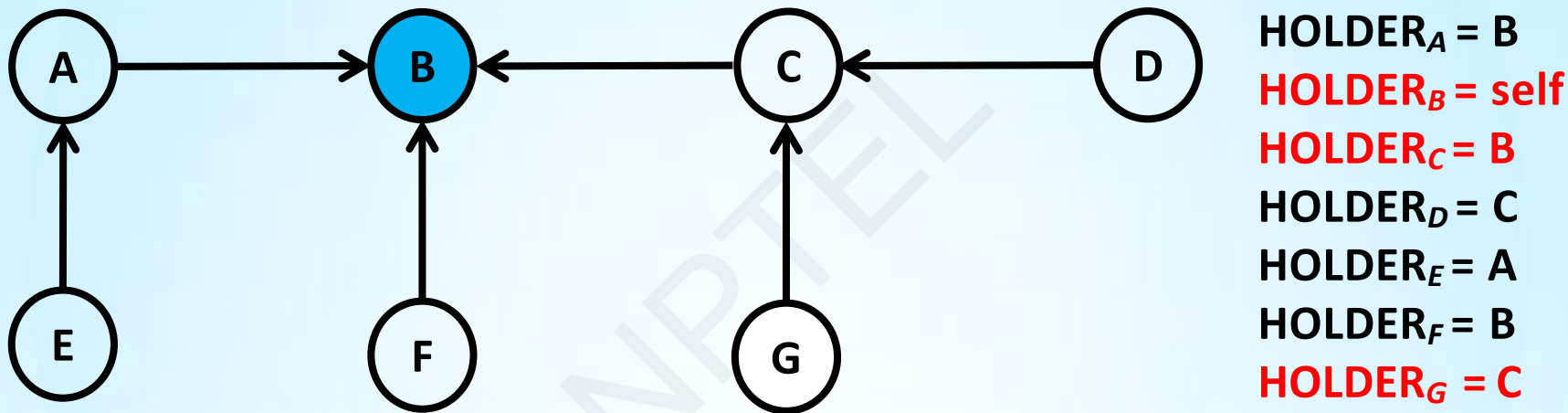
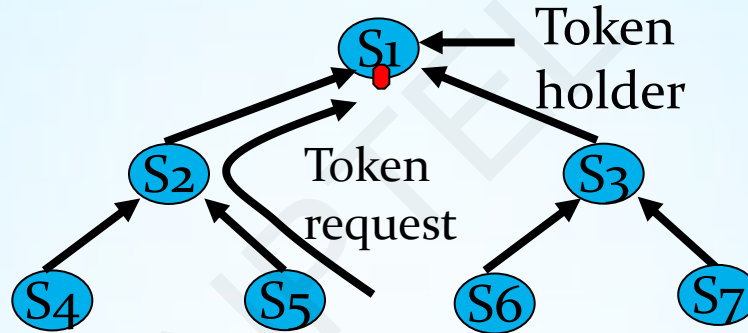


Figure 9.3 Tree with logically directed edges, all pointing in a direction towards node B - the new privileged node.

Raymond's Algorithm: Example (2)

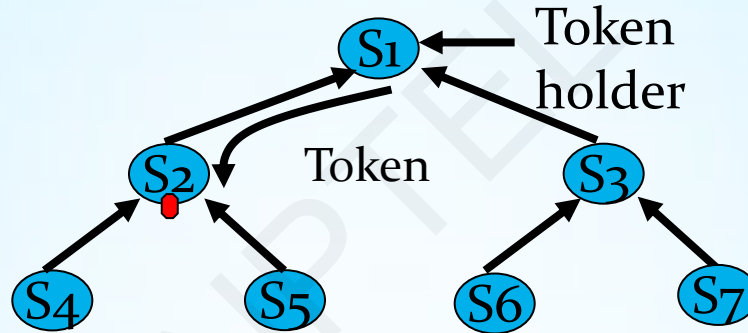
Step 1:



Site S5 sends token request

Raymond's Algorithm: Example (2)

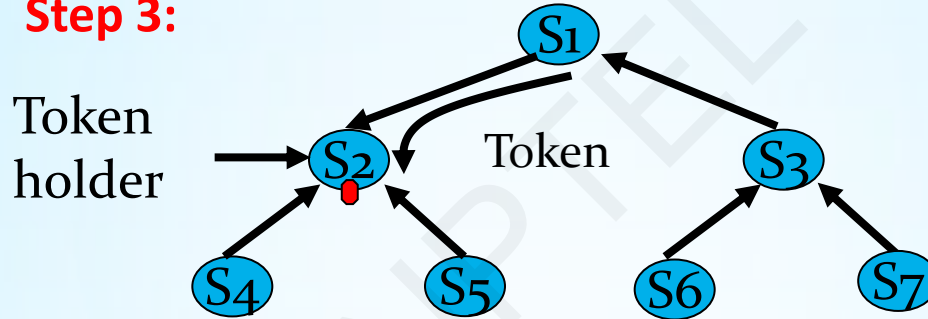
Step 2:



Site S1 sends token to S2

Raymond's Algorithm: Example (2)

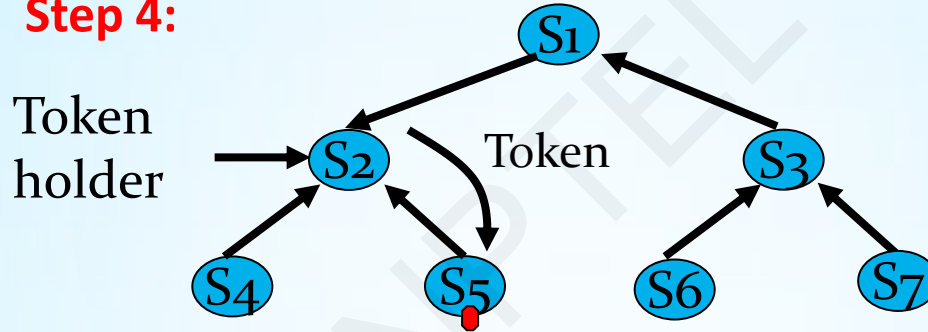
Step 3:



Holder of S1 is changed to S2

Raymond's Algorithm: Example (2)

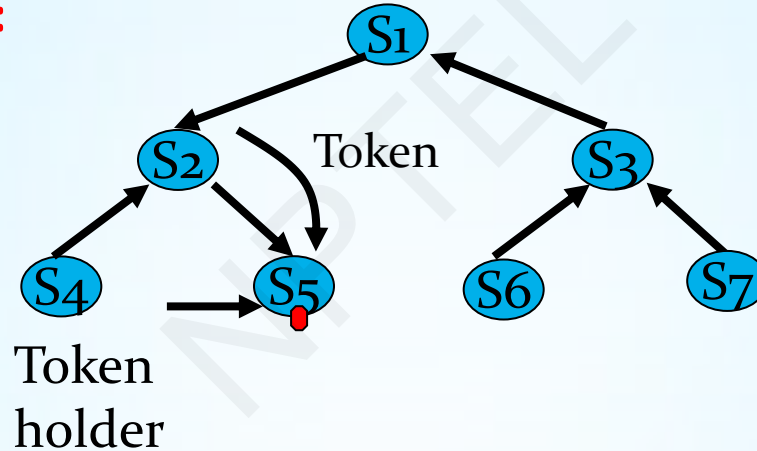
Step 4:



S2 sends token to S5

Raymond's Algorithm: Example (2)

Step 5:



**Holder of S2 is changed to S5 and
S5 becomes the new token holder**

Data Structures

- Each node to maintains the following variables:

Variable Name	Possible Values	Comments
HOLDER	“self” or the identity of one of the immediate neighbours.	Indicates the location of the privileged node in relation to the current node.
USING	True or false.	Indicates if the current node is executing the critical section.
REQUEST_Q	A FIFO queue that could contain “self” or the identities of immediate neighbors as elements.	The REQUEST_Q of a node consists of the identities of those immediate neighbors that have requested for privilege but have not yet been sent the privilege .
ASKED	True or false.	Indicates if node has sent a request for the privilege.

Data Structures

- The value “**self**” is placed in **REQUEST_Q** if the node makes a request for the privilege for its own use.
- The **maximum size** of **REQUEST_Q** of a node is the number of **immediate neighbors + 1** (for “self”).
- **ASKED** prevents the sending of duplicate requests for privilege, and also makes sure that the **REQUEST_Qs** of the various nodes do not contain any duplicate elements.

The Algorithm

- The algorithm consists of the following routines:

(i) **ASSIGN_PRIVILEGE** (ii) **MAKE_REQUEST**

(i) **ASSIGN_PRIVILEGE**: This is a routine sends a **PRIVILEGE** message. A privileged node sends a **PRIVILEGE** message if

- it holds the privilege but is not using it,
- its **REQUEST_Q** is not empty, and
- the element at the head of its **REQUEST_Q** is not “self.”

(i) ASSIGN_PRIVILEGE

- A situation where “self” is at the head of **REQUEST_Q** may occur immediately after a node receives a **PRIVILEGE** message.
- The node will enter into the critical section after removing “self” from the head of **REQUEST_Q**. If the id of another node is at the head of **REQUEST_Q**, then it is removed from the queue and a **PRIVILEGE** message is sent to that node.
- Also, the variable **ASKED** is set to false since the currently privileged node will not have sent a request for the **PRIVILEGE** message.

(ii) MAKE_REQUEST

This is a routine sends a **REQUEST** message. An unprivileged node sends a **REQUEST** message if

- it does not hold the privilege,
- its **REQUEST_Q** is not empty, i.e., it requires the privilege for itself, or on behalf of one of its immediate neighboring nodes, and
- it has not sent a **REQUEST** message already.

(ii) MAKE_REQUEST

- The variable **ASKED** is set to true to reflect the sending of the **REQUEST** message. The **MAKE_REQUEST** routine makes no change to any other variables.
- The variable **ASKED** will be true at a node when it has sent **REQUEST** message to an immediate neighbor and has not received a response.
- A node does not send any **REQUEST** messages, if **ASKED** is true at that node. Thus the variable **ASKED** makes sure that unnecessary **REQUEST** messages are not sent from the unprivileged node.
- This makes the **REQUEST_Q** of any node bounded, even when operating under heavy load.

Events

- Below we show four events that constitute the algorithm.

Event	Algorithm Functionality
A node wishes to execute critical section.	Enqueue(REQUEST_Q, self); ASSIGN_PRIVILEGE; MAKE_REQUEST
A node receives a REQUEST message from one of its immediate neighbors X.	Enqueue(REQUEST_Q, X); ASSIGN_PRIVILEGE; MAKE_REQUEST
A node receives a PRIVILEGE message.	HOLDER := self; ASSIGN_PRIVILEGE; MAKE_REQUEST
A node exits the critical section.	USING := false; ASSIGN_PRIVILEGE; MAKE_REQUEST

Events

A node wishes critical section entry:

- If it is the privileged node, the node could enter the critical section using the **ASSIGN_PRIVILEGE** routine. Otherwise, it sends a **REQUEST** message using the **MAKE_REQUEST** routine.

A node receives a REQUEST message from one of its immediate neighbors:

- If this node is the current **HOLDER**, it may send the **PRIVILEGE** to a requesting node using the **ASSIGN_PRIVILEGE** routine. Otherwise, it forwards the request using the **MAKE_REQUEST** routine.

Events

A node receives a PRIVILEGE message:

- The **ASSIGN_PRIVILEGE** routine could result in the execution of the critical section at the node, or may forward the privilege to another node. After the
- privilege is forwarded, the **MAKE_REQUEST** routine could send a **REQUEST** message to reacquire the privilege, for a pending request at this node.

A node exits the critical section:

- On exit from the critical section, this node may pass the privilege on to a requesting node using the **ASSIGN_PRIVILEGE** routine. It may then use the
- **MAKE_REQUEST** routine to get back the privilege, for a pending request at this node.

Events

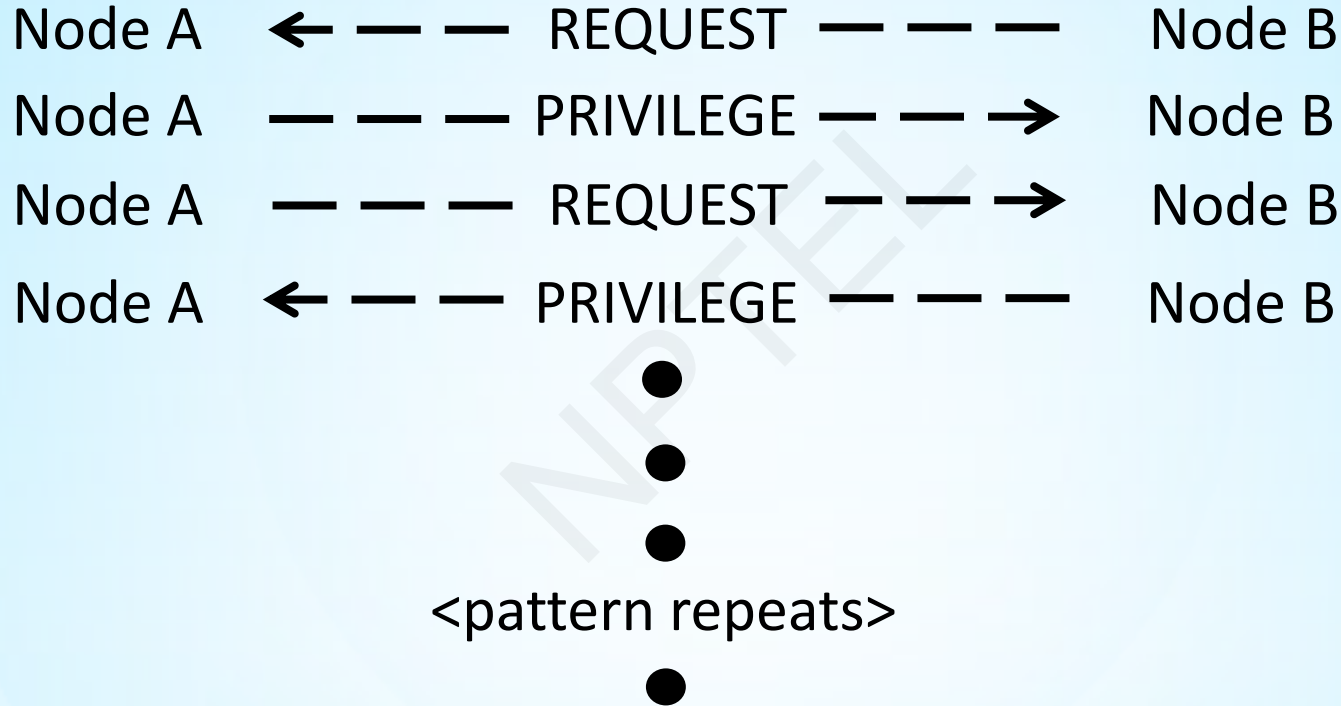


Figure 9.4: Logical pattern of message flow between neighboring nodes A and B.

Message Overtaking

- This algorithm does away with the use of sequence numbers. The algorithm works such that message flow between any two neighboring nodes sticks to a logical pattern as shown in the Figure 9.4.
- If at all message overtaking occurs between the nodes A and B, it can occur when a **PRIVILEGE** message is sent from node A to node B, which is then very closely followed by a **REQUEST** message from node A to node B.
- Such a message overtaking will not affect the operation of the algorithm.
- If node B receives the **REQUEST** message from node A before receiving the **PRIVILEGE** message from node A, A's request will be queued in **REQUEST_Q_B**. Since B is not a privileged node, it will not be able to send a privilege to node A in reply.

Message Overtaking

- When node B receives the **PRIVILEGE** message from A after receiving the **REQUEST** message, it could enter the critical section or could send a **PRIVILEGE** message to an immediate neighbor at the head of **REQUEST_Q_B**, which need not be node A. So message overtaking does not affect the algorithm.

Correctness

The algorithm provides the following guarantees:

- Mutual exclusion is guaranteed
- Deadlock is impossible
- Starvation is impossible

Mutual Exclusion

- The algorithm ensures that at any instant of time, not more than one node holds the privilege.
- Whenever a node receives a **PRIVILEGE** message, it becomes privileged. Similarly, whenever a node sends a **PRIVILEGE** message, it becomes unprivileged.
- Between the instants one node becomes unprivileged and another node becomes privileged, there is no privileged node. Thus, there is at most one privileged node at any point of time in the network.

Deadlock is Impossible

When the critical section is free, and one or more nodes want to enter the critical section but are not able to do so, a deadlock may occur. This could happen due to any of the following scenarios:

- The privilege cannot be transferred to a node because no node holds the privilege.
- The node in possession of the privilege is unaware that there are other nodes requiring the privilege.
- The **PRIVILEGE** message does not reach the requesting unprivileged node.

Deadlock is Impossible

- The **scenario 1** can never occur in this algorithm because nodes do not fail and messages are not lost.
- The logical pattern established using **HOLDER** variables ensures that a node that needs the privilege sends a **REQUEST** message either to a node holding the privilege or to a node that has a path to a node holding the privilege. Thus **scenario 2** can never occur.
- The series of **REQUEST** messages are enqueued in the **REQUEST_Qs** of various nodes such that the **REQUEST_Qs** of those nodes collectively provide a logical path for the transfer of the PRIVILEGE message from the privileged node to the requesting unprivileged nodes. So scenario 3 can never occur.

Starvation is Impossible

- When a node A holds the privilege, and another node B requests for the privilege, the identity of B or the id's of proxy nodes for node B will be present in the **REQUEST_Qs** of various nodes in the path connecting the requesting node to the currently privileged node.
- So depending upon the position of the id of node B in those **REQUEST_Qs**, node B will sooner or later receive the privilege.
- Thus once node B's **REQUEST** message reaches the privileged node A, node B, is sure to receive the privilege.

Cost and Performance Analysis

- In the **worst-case**, the algorithm requires **$(2 * \text{longest path length of the tree})$ messages** per critical section entry.
- This happens when the privilege is to be passed between nodes at either ends of the longest path of the minimal spanning tree.
- The **worst possible** network topology for this algorithm is **where all nodes are arranged in a straight line** and the longest path length will be $N - 1$, and thus the algorithm will exchange **$2 * (N - 1)$ messages** per CS execution.
- However, if all nodes generate **equal number of REQUEST messages** for the privilege, the **average number of messages** needed per critical section entry will be **approximately $2N/3$** because the **average distance between a requesting node and a privileged node** is **$(N + 1)/3$** .

Cost and Performance Analysis

- The **best topology** for the algorithm is the **radiating star topology**. The **worst case cost** of this algorithm for this topology is $O(\log_{K-1} N)$.
- Trees with higher fan-outs are preferred over radiating star topologies. The **longest path length** of such trees is typically $O(\log N)$. Thus, on an average, this algorithm involves the exchange of **$O(\log N)$ messages** per critical section execution.
- **Under heavy load**, the algorithm exhibits an interesting property: “As the number of nodes requesting for the privilege increases, the number of messages exchanged per critical section entry decreases.”
- **In heavy load**, the **algorithm requires** exchange of **only four messages** per CS execution.

Comparison of Distributed Mutual Exclusion Algorithms

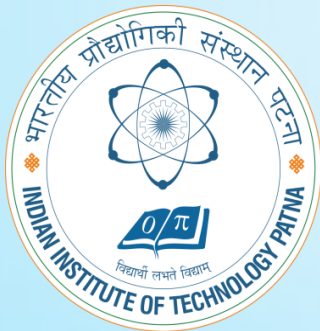
Non-Token	Response Time	Synchronization Delay	Messages (Low Load)	Messages (High Load)	Concept
Lamport	$2T+E$	T	$3(N-1)$	$3(N-1)$	Give Priority with Time Stamp
Ricart-Agrawala	$2T+E$	T	$2(N-1)$	$2(N-1)$	Uses Implicit Release Message strategy.
Quorum	Response Time	Synchronization Delay	Messages (Low Load)	Messages (High Load)	Concept
Maekawa	$2T+E$	$2T$	$3 \cdot \sqrt{N}$	$5 \cdot \sqrt{N}$	Uses Theory of projective planes for quorums.
Agarwal-El Abbadi	$2T+E$	$2T$	$\log N$	$\log N$	Uses Tree structured quorums.
Token	Response Time	Synchronization Delay	Messages (Low Load)	Messages (High Load)	Concept
Suzuki-Kasami	$2T+E$	T	N	N	Token as a privilege message.
Raymond Tree	$T(\log N)+E$	$T(\log N) / 2$	$\log N$	4	Minimal Spanning Tree and unrooted tree structure

Conclusion

- **Mutual exclusion** is a fundamental problem in distributed computing systems, where concurrent access to a shared resource or data is serialized.
- In **distributed mutual exclusion algorithms**, we have discussed three types:
 - (i) **Non-Token-based approach,**
 - (ii) **Quorum based approach and**
 - (iii) **Token-based approach**
- In upcoming lecture, we will discuss about the 'consensus and agreement algorithms'.

Lecture: 10

Consensus and Agreement Algorithms



Rajiv Misra

Dept. of Computer Science &
Engineering

Indian Institute of Technology Patna

rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have discussed about **‘Mutual exclusion algorithms’** for **‘distributed computing systems’** such as:
 - (i) Non-Token based approach,
 - (ii) Quorum based approach and
 - (iii) Token based approach

Preface

Content of this Lecture:

- In this lecture, we will discuss about **'Consensus and Agreement Algorithms'**.
- This lecture first covers different forms of the **'consensus problem'** then gives an overview of what forms of consensus are solvable under different failure models and different assumptions on the synchrony/asynchrony.
- Also covers agreement in the category of:
 - (i) Synchronous message-passing systems with failures and**
 - (ii) Asynchronous message-passing systems with failures.**

Introduction

- **Agreement among the processes** in a distributed system is a fundamental requirement for a wide range of applications.
 - Many forms of coordination require the processes to exchange information to negotiate with one another and eventually reach a common understanding or agreement, before taking application-specific actions.
 - **A classical example is that of the commit decision in database systems**, wherein the processes collectively decide whether to commit or abort a transaction that they participate in.
- In this lecture, we will study the feasibility of designing algorithms to reach agreement under various system models and failure models, and, where possible, examine some representative algorithms to reach agreement.

Classification of Faults: Overview

- **Based on components that failed**
 - Program / process
 - Processor / machine
 - Link
 - Storage
- **Based on behavior of faulty component**
 - Crash – just halts
 - Fail stop – crash with additional conditions
 - Omission – fails to perform some steps
 - Byzantine – behaves arbitrarily
 - Timing – violates timing constraints

Classification of Tolerance: Overview

- **Types of tolerance:**

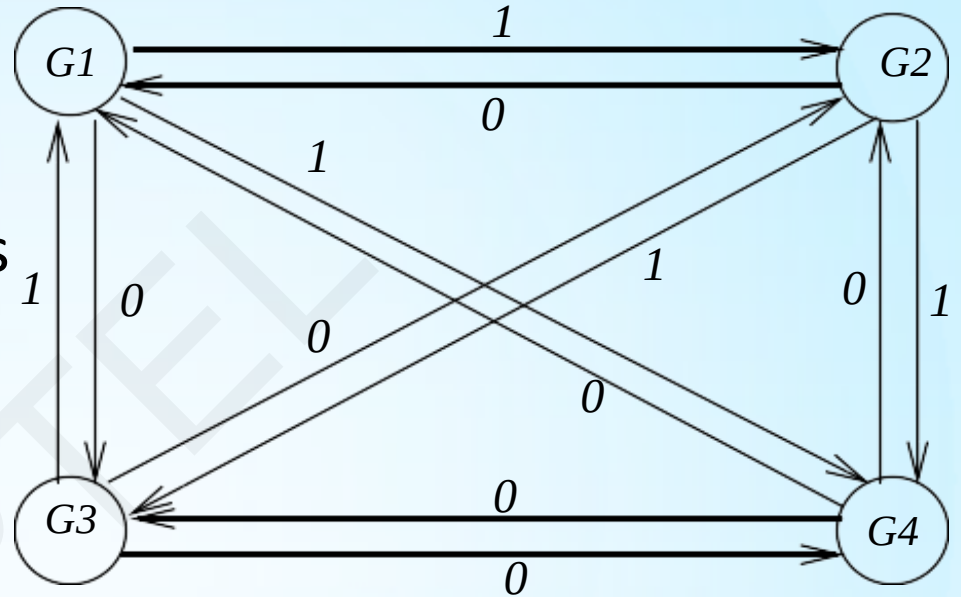
- **Masking** – System always behaves as per specifications even in presence of faults.
- **Non-masking** – System may violate specifications in presence of faults. Should at least behave in a well-defined manner.

- **Fault tolerant system should specify:**

- Class of faults tolerated
- What tolerance is given from each class

Assumptions

1. Failure models
2. Synchronous/ Asynchronous communication
3. Network connectivity
4. Sender identification
5. Channel reliability
6. Authenticated vs. Non-authenticated messages
7. Agreement variable



1) Failure models

- A failure model specifies the manner in which the component(s) of the system may fail.
- There exists a rich class of **well-studied failure models**. The various process failure models are: (i) Fail-stop, (ii) Crash, (iii) Receive omission, (iv) Send omission, (v) General omission, and (vi) Byzantine or malicious failures
- Among the n processes in the system, at most f processes can be faulty. A faulty process can behave in any manner allowed by the failure model assumed.

Type of Process Failure Models

- i. **Fail-stop:** In this model, a properly functioning process may fail by stopping execution from some instant thenceforth. Additionally, other processes can learn that the process has failed.
- ii. **Crash:** In this model, a properly functioning process may fail by stopping to function from any instance thenceforth. Unlike the fail-stop model, other processes do not learn of this crash.
- iii. **Receive omission:** A properly functioning process may fail by intermittently receiving only some of the messages sent to it, or by crashing.
- iv. **Send omission:** A properly functioning process may fail by intermittently sending only some of the messages it is supposed to send, or by crashing.

Contd...

- v. **General omission:** A properly functioning process may fail by exhibiting either or both of send omission and receive omission failures.
- vi. **Byzantine or malicious failure:** In this model, a process may exhibit any arbitrary behavior and no authentication techniques are applicable to verify any claims made.

2) Synchronous/Asynchronous Computation

-Synchronous Computation:

- i. Processes run in lock step manner[Process receives a message sent to it earlier, performs computation and sends a message to other process.
- ii. Step of Synchronous computation is called round

-Asynchronous Computation:

- i. Computation does not proceed in lock step.
- ii. Process can send receive messages and perform computation at any time.

3) Network connectivity

The system has **full logical connectivity**, i.e., each process can communicate with any other by direct message passing.

4) Sender identification

A process that receives a message always **knows the identity** of the sender process.

5) Channel reliability

The **channels are reliable**, and only the processes may fail (under one of various failure models). This is a simplifying assumption in our study.

6) Authenticated vs. Non-authenticated messages

- In this study, we will be **dealing only with unauthenticated messages**.
- With **unauthenticated messages**, when a faulty process relays a message to other processes, (i) it can forge the message and claim that it was received from another process, and (ii) it can also tamper with the contents of a received message before relaying it.
- Using **authentication** via techniques such as digital signatures, it is easier to solve the agreement problem because, if some process forges a message or tampers with the contents of a received message before relaying it, the recipient can detect the forgery or tampering.

7) Agreement variable

- The agreement variable **may be boolean or multivalued**, and need not be an integer.
- When studying some of the more complex algorithms, we will use a boolean variable.
- This simplifying assumption does not affect the results for other data types, but helps in the abstraction while presenting the algorithms.

Performance Aspects of Agreement Protocols

Few Performance Metrics are as follows:

- (i) Time:** No of rounds needed to reach an agreement
- (ii) Message Traffic:** Number of messages exchanged to reach an agreement.
- (iii) Storage Overhead:** Amount of information that needs to be stored at processors during execution of the protocol.

Problem Specifications

1. Byzantine Agreement Problem (single source has an initial value)

Agreement: All non-faulty processes must agree on the same value.

Validity: If the source process is non-faulty, then the agreed upon value by all the non-faulty processes must be the same as the initial value of the source.

Termination: Each non-faulty process must eventually decide on a value.

2. Consensus Problem (all processes have an initial value)

Agreement: All non-faulty processes must agree on the same (single) value.

Validity: If all the non-faulty processes have the same initial value, then the agreed upon value by all the non-faulty processes must be that same value.

Termination: Each non-faulty process must eventually decide on a value.

Contd...

3. Interactive Consistency Problem (all processes have an initial value)

Agreement: All non-faulty processes must agree on the same array of values $A[v_1 \dots v_n]$.

Validity: If process i is non-faulty and its initial value is v_i , then all non-faulty processes agree on v_i as the i th element of the array A . If process j is faulty, then the non-faulty processes can agree on any value for $A[j]$.

Termination: Each non-faulty process must eventually decide on the array A .

Equivalence of the Problems

- The three problems defined above are equivalent in the sense that a solution to any one of them can be used as a solution to the other two problems. This equivalence can be shown using a reduction of each problem to the other two problems.
- If problem A is reduced to problem B, then a solution to problem B can be used as a solution to problem A in conjunction with the reduction.
- Formally, the **difference between the agreement problem and the consensus problem** is that, in the agreement problem, a single process has the initial value, whereas in the consensus problem, all processes have an initial value.
- However, the two terms are used interchangeably in much of the literature and hence we shall also use the terms interchangeably.

Overview of Results

- **Table 10.1** gives an overview of the results and lower bounds on solving the consensus problem under different assumptions.
- It is worth understanding the relation between the consensus problem and the problem of attaining common knowledge of the agreement value. For the “no failure” case, consensus is attainable.
- Further, in a synchronous system, common knowledge of the consensus value is also attainable, whereas in the asynchronous case, concurrent common knowledge of the consensus value is attainable.

Overview of Results

Failure mode	Synchronous system (message-passing and shared memory)	Asynchronous system (message-passing and shared memory)
No failure	agreement attainable; common knowledge also attainable	agreement attainable; concurrent common knowledge attainable
Crash failure	agreement attainable $f < n$ processes $\Omega(f + 1)$ rounds	agreement not attainable
Byzantine failure	agreement attainable $f \leq \lfloor (n - 1)/3 \rfloor$ Byzantine processes $\Omega(f + 1)$ rounds	agreement not attainable

Table 10.1: Overview of results on agreement. f denotes number of failure-prone processes. n is the total number of processes.

In a failure-free system, consensus can be attained in a straightforward manner

Contd...

- **Consensus is not solvable in asynchronous systems** even if one process can fail by crashing.
- **Figure 10.1** shows further how asynchronous message-passing systems and shared memory systems deal with trying to solve consensus.

Solvable Variants of the Consensus Problem in Asynchronous Systems

Circumventing the impossibility results for consensus in asynchronous systems

Message-passing

k set consensus

epsilon- consensus

Renaming

Reliable broadcast

Shared memory

k set consensus

epsilon- consensus

Renaming

*using atomic registers and
atomic snapshot objects
constructed from atomic
registers*

Consensus

*using more powerful
objects than atomic registers. This
is the study of universal objects and
universal constructions.*

Weaker Consensus Problems in Asynchronous System

Consensus Problem	Description
Terminating reliable broadcast	It states that a correct process always gets a message even if the sender crashes while sending. If the sender crashes while sending the message, the message may be a null message but it must be delivered to each correct process.
k-set consensus	It is solvable as long as the number of crash failures f is less than the parameter k . The parameter k indicates that the non-faulty processes agree on different values, as long as the size of the set of values agreed upon is bounded by k .
Approximate agreement	Like k -set consensus, approximate agreement also assumes the consensus value is from a multi-valued domain. However, rather than restricting the set of consensus values to a set of size k , ϵ -approximate agreement requires that the agreed upon values by the non-faulty processes be within ϵ of each other.
Renaming problem	It requires the processes to agree on necessarily distinct values.
Reliable broadcast	A weaker version of reliable terminating broadcast(RTB), namely reliable broadcast, in which the termination condition is dropped, is solvable under crash failures.

Contd...

- To circumvent the impossibility result, weaker variants of the consensus problem are defined in **Table 10.2**.
- The overheads given in this table are for the algorithms described.

Some Solvable Variants of the Consensus Problem in Asynchronous Systems

Solvable Variants	Failure model and overhead	Definition
Reliable broadcast	crash failures, $n > f$ (MP)	Validity, Agreement, Integrity conditions
k-set consensus	crash failures. $f < k < n$. (MP and SM)	size of the set of values agreed upon must be less than k
ϵ -agreement	crash failures $n \geq 5f + 1$ (MP)	values agreed upon are within ϵ of each other
Renaming	up to f fail-stop processes, $n \geq 2f + 1$ (MP) Crash failures $f \leq n - 1$ (SM)	select a unique name from a set of names

Table 10.2: Some solvable variants of the agreement problem in asynchronous system. The overhead bounds are for the given algorithms, and not necessarily tight bounds for the problem. Here MP- Message Passing, SM- Shared Memory

Agreement in Synchronous Message-Passing Systems with Failures

1. Consensus Algorithm for Crash Failures (Message Passing, Synchronous System)

- **Algorithm 10.1** gives a consensus algorithm for n processes, where up to f processes, where $f < n$, may fail in the fail-stop model.
- Here, the consensus variable x is integer-valued. Each process has an initial value x_i .
- If up to f failures are to be tolerated, then the algorithm has $f + 1$ rounds.
- In each round, a process i sends the value of its variable x_i to all other processes
- if that value has not been sent before.
- Of all the values received within the round and its own value x_i at the start of the round, the process takes the minimum, and updates x_i . After $f + 1$ rounds, the local value x_i is guaranteed to be the consensus value.

Consensus Algorithm for Crash Failures

(global constants)

integer: f ; // maximum number of crash failures tolerated

(local variables)

integer: $x \leftarrow$ local value;

(1) Process P_i ($1 \leq i \leq n$) executes the Consensus algorithm for up to f crash failures:

(1a) **for** round **from** 1 **to** $f + 1$ **do**

(1b) **if** the current value of x has not been broadcast **then**

(1c) **broadcast**(x);

(1d) $y_j \leftarrow$ value (if any) received from process j in this round;

(1e) $x \leftarrow \min(x, y_j)$;

(1f) **output** x as the consensus value.

Algorithm 10.1 Consensus with up to f fail-stop processes in a system of n processes, $n > f$. Code shown is for process P_i $1 \leq i \leq n$

Complexity

- There are $f+1$ rounds, where $f < n$.
- The number of messages is **at most $O(n^2)$ in each round**, and each message has one integer.
- Hence the total number of messages is **$O((f+1) \cdot n^2)$**

2. Consensus Algorithm for Byzantine Failure (Message Passing, Synchronous Systems)

Model :

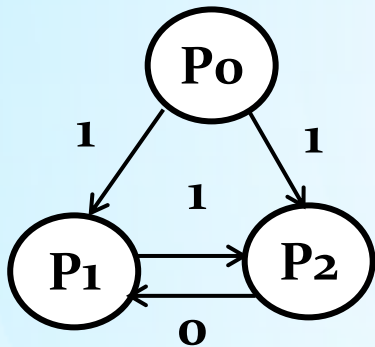
- Total of n processes, at most f of which can be faulty
- Reliable communication medium
- Fully connected
- Receiver always knows the identity of the sender of a message
- Byzantine faults
- Synchronous system: In each round, a process receives messages, performs computation, and sends messages.

Solution for Byzantine Agreement Problem

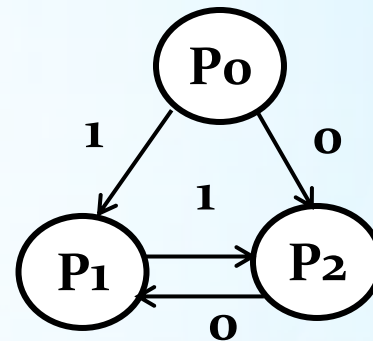
- The solution of Byzantine Agreement Problem is first defined and solved by *lamport*.
- Pease showed* that in a fully connected network, it is impossible to reach an agreement if number of faulty processes ' f ' exceeds $(n-1)/3$ where n is number of processes

Byzantine agreement can not be reached among three processes if one process is faulty

P_0 is Non-Faulty



P_0 is Faulty



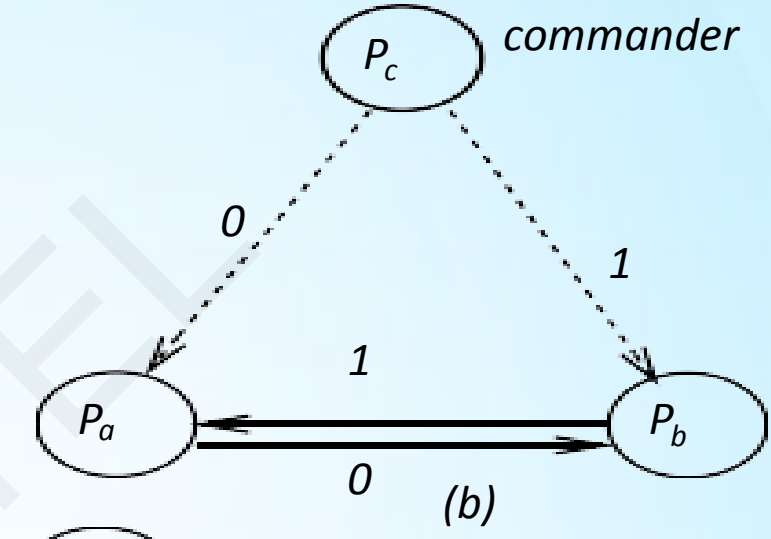
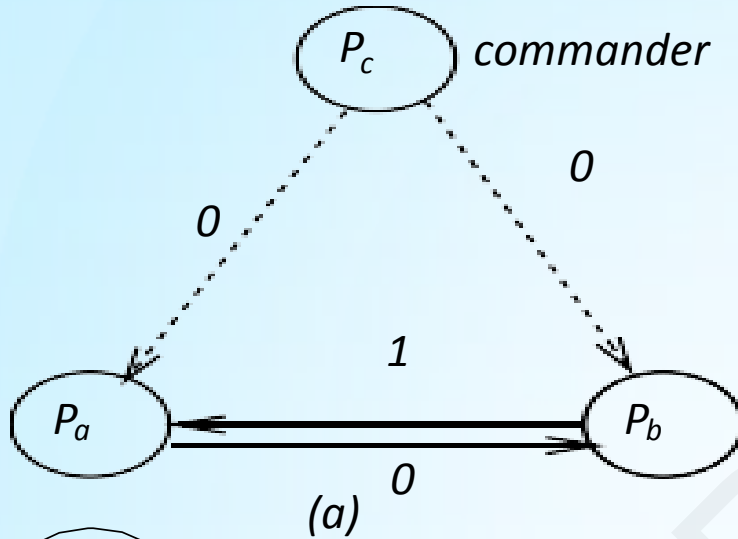
Note: Here P_0 is a source process

Upper bound on Byzantine processes

In a system of n processes, the Byzantine agreement problem (as also the other variants of the agreement problem) can be solved in a synchronous only if the number of Byzantine processes f is such that $f \leq \lfloor (n - 1)/3 \rfloor$

Upper Bound on Byzantine Processes

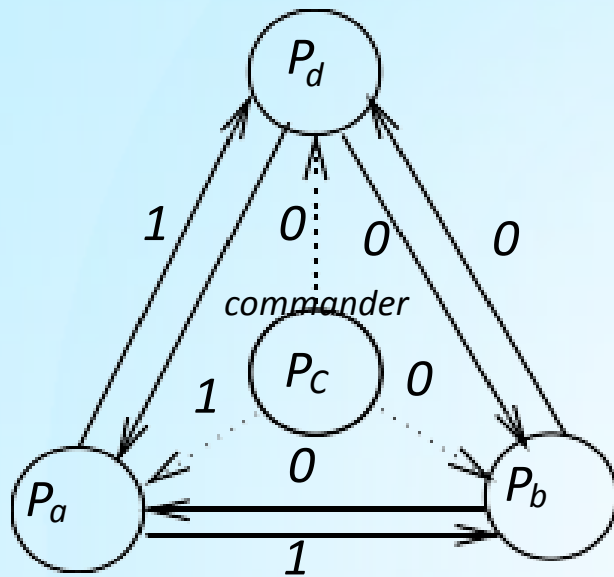
Agreement impossible when $f = 1, n = 3$.



Taking simple majority decision does not help because loyal commander P_a cannot distinguish between the possible scenarios (a) and (b); hence does not know which action to take.

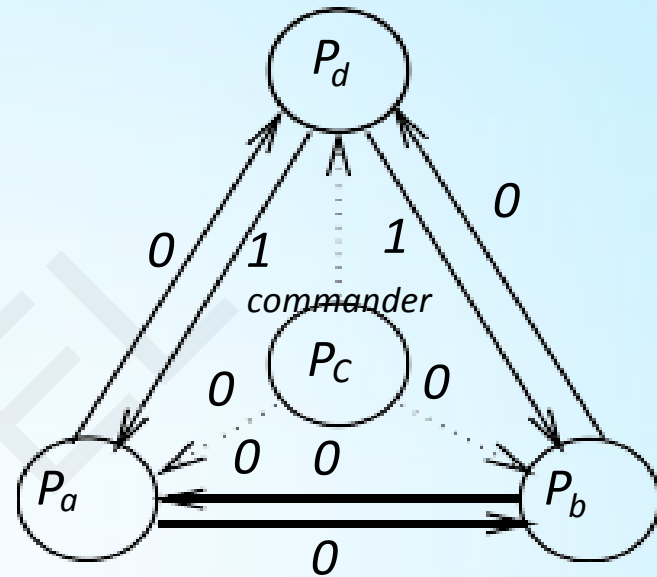
Proof using induction that problem solvable if $f \leq \lfloor (n - 1)/3 \rfloor$

Consensus Solvable when $f = 1, n = 4$



.....> first round exchange

○ malicious process



————> second round exchange

○ correct process

- There is no ambiguity at any loyal commander, when taking majority decision
- Majority decision is over 2nd round messages, and 1st round message received directly from commander-in-chief process.

Lamport-Shostak-Pease Algorithm

This algorithm also known as *Oral Message Algorithm OM(f)*
where f is the number of faulty processes
' n ' = Number of processes and $n \geq 3f+1$

Algorithm is Recursively defined as follows:

Algorithm OM(0)

1. Source process sends its values to each other process
2. Each process uses the value it receives from the source.
[If no value is received default value 0 is used]

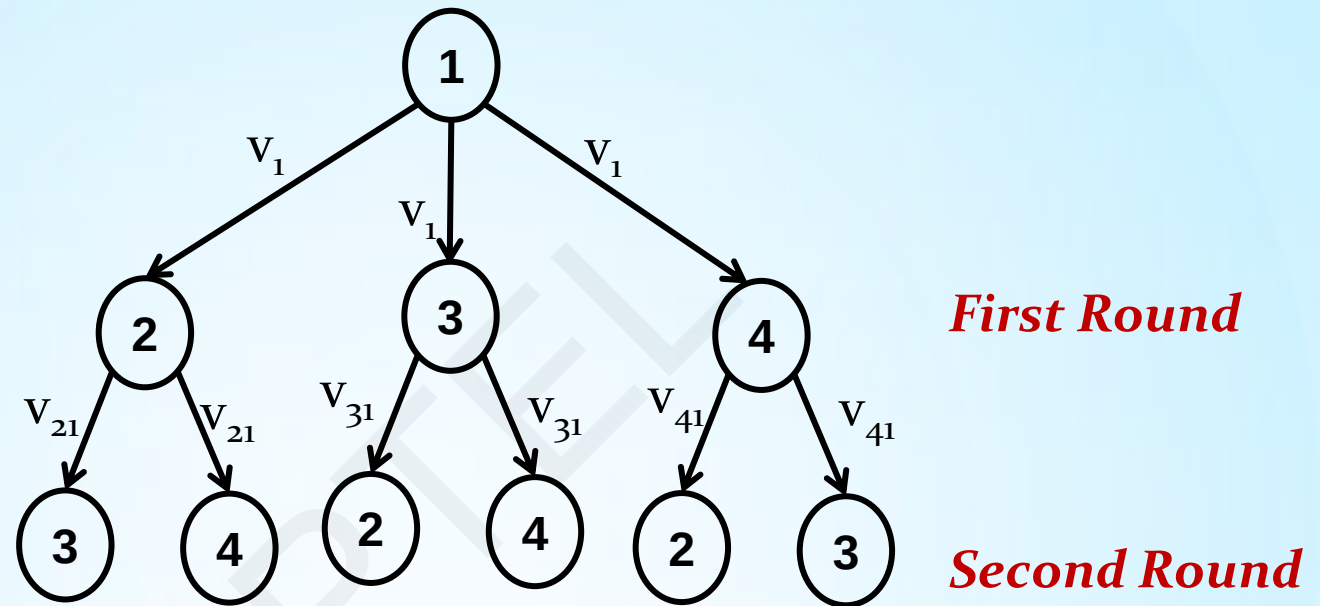
Contd...

Algorithm OM(f), $f > 0$

1. The source process sends its value to each other process.
2. For each i , let v_i be the value process i receives from source.
[Default value 0 if no value received]
3. Process i acts as the new source and initiates **Algorithm OM($f-1$)** where it sends the value v_i to each of the $n-2$ other processes.
4. For each i and j (not i), let v_j be the value process i received from process j in STEP 3. Process i uses the value **majority** ($v_1, v_2, \dots, v_{n-1}$).

“The function majority($v_1, v_2, \dots, v_{n-1}$) computes the majority value if exists otherwise it uses default value 0.”

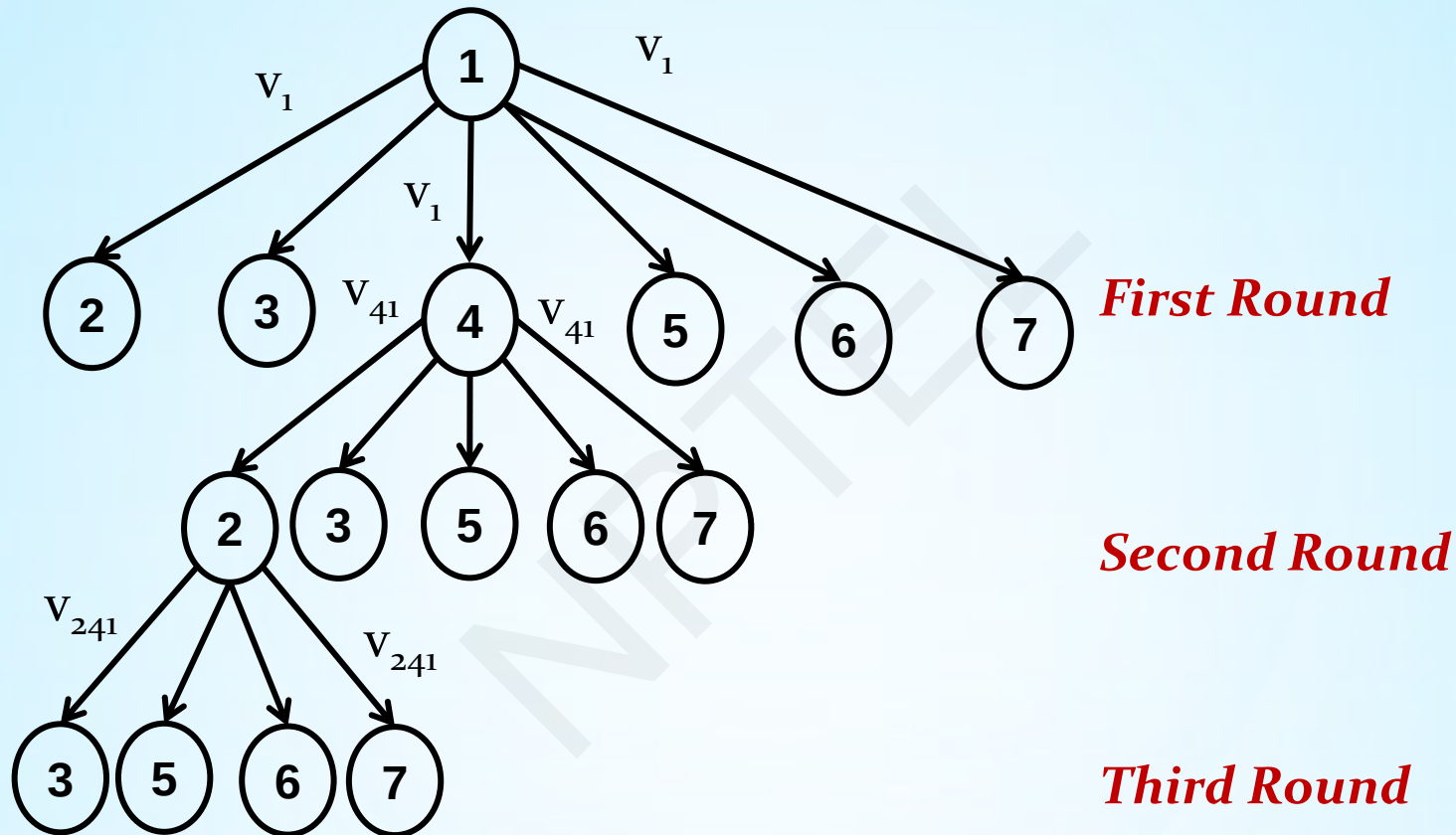
(i) Solving the Byzantine agreement, for $f = 1$ and $n = 4$



Round number	A message has already visited	Aims to tolerate these many failures	And each message gets sent to	Total number of messages in round
1	1	1	$4 - 1 = 3$	$4 - 1 = 3$
2	2	$1 - 1 = 0$	$4 - 2 = 2$	$(4 - 1) \cdot (4 - 2) = 3 \cdot 2$

- Number of messages for agreement on one value is: $3 + 2 \cdot 3 = 9$ messages

(ii) Solving the Byzantine agreement, for $f = 2$ and $n = 7$



Contd...

Round number	A message has already visited	Aims to tolerate these many failures	And each message gets sent to	Total number of messages in round
1	1	2	$7 - 1 = 6$	$7 - 1 = 6$
2	2	$2 - 1 = 1$	$7 - 2 = 5$	$(7 - 1) \cdot (7 - 2) = 6.5$
3	3	$2 - 2 = 0$	$7 - 3 = 4$	$(7 - 1) \cdot (7 - 2) \cdot (7 - 3) = 6.5.4$

- Number of messages for agreement on one value is: $6 + 6.5 + 6.5.4 = 156$ messages

(iii) Solving the Byzantine agreement, for $f = 3$ and $n = 10$

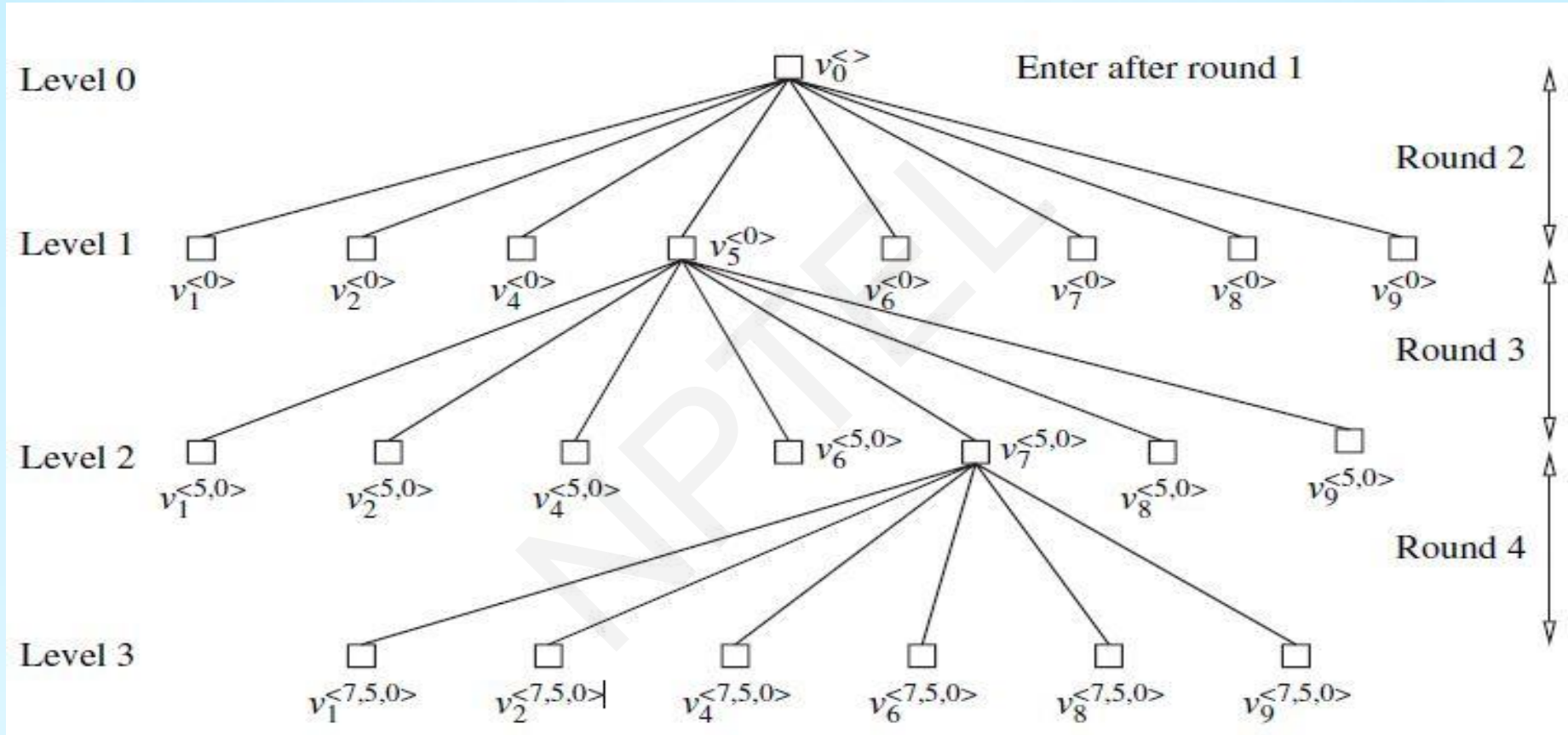


Fig. Local tree at P_3 for solving the Byzantine agreement, for $n = 10$ and $f = 3$, commander is P_0 . Only one branch of the tree is shown for simplicity.

Contd...

Round number	A message has already visited	Aims to tolerate these many failures	And each message gets sent to	Total number of messages in round
1	1	3	$10 - 1 = 9$	$10 - 1 = 9$
2	2	$3 - 1 = 2$	$10 - 2 = 8$	$(10 - 1) \cdot (10 - 2) = 9.8$
3	3	$3 - 2 = 1$	$10 - 3 = 7$	$(10 - 1) \cdot (10 - 2) \cdot (10 - 3) = 9.8.7$
4	4	$3 - 3 = 0$	$10 - 4 = 6$	$(10 - 1) \cdot (10 - 2) \cdot (10 - 3) \cdot (10 - 4) = 9.8.7.6$

- Number of messages for agreement on one value is: $9 + 9.8 + 9.8.7 + 9.8.7.6 = 3609$ messages

Relationship between # Messages and Rounds

Round number	A message has already visited	Aims to tolerate these many failures	And each message gets sent to	Total number of messages in round
1	1	f	$n - 1$	$n - 1$
2	2	$f - 1$	$n - 2$	$(n - 1) \cdot (n - 2)$
...	...	...	...	...
x	x	$(f + 1) - x$	$n - x$	$(n - 1)(n - 2) \dots (n - x)$
$x + 1$	$x + 1$	$(f + 1) - x - 1$	$n - x - 1$	$(n - 1)(n - 2) \dots (n - x - 1)$
$f + 1$	$f + 1$	0	$n - f - 1$	$(n - 1)(n - 2) \dots (n - f - 1)$

Table: Relationships between messages and rounds in the Oral Messages algorithm for Byzantine agreement.

Complexity: $f + 1$ rounds, exponential amount of space, and $(n - 1) + (n - 1)(n - 2) + \dots + (n - 1)(n - 2) \dots (n - f - 1)$ messages

Agreement in Asynchronous Message-Passing Systems with Failures

Impossibility Result for the Consensus Problem

Fischer-Lynch-Paterson (FLP) Impossibility Result (By M. Fischer, N. Lynch, and M. Paterson, April 1985)

- **Fischer et al.** showed a fundamental result on the impossibility of reaching agreement in an asynchronous (message-passing) system.
- It states that it is “*Impossible to reach consensus in an asynchronous message passing system even if a single process has a crash failure*”
- This result, popularly known as the **FLP impossibility result**, has a significant impact on the field of designing distributed algorithms in a failure-susceptible system.

Weaker Versions of Consensus Problem

Consensus Problem	Description
Terminating reliable broadcast	It states that a correct process always gets a message even if the sender crashes while sending. If the sender crashes while sending the message, the message may be a null message but it must be delivered to each correct process.
k-set consensus	It is solvable as long as the number of crash failures f is less than the parameter k . The parameter k indicates that the non-faulty processes agree on different values, as long as the size of the set of values agreed upon is bounded by k .
Approximate agreement	Like k -set consensus, approximate agreement also assumes the consensus value is from a multi-valued domain. However, rather than restricting the set of consensus values to a set of size k , ϵ -approximate agreement requires that the agreed upon values by the non-faulty processes be within ϵ of each other.
Renaming problem	It requires the processes to agree on necessarily distinct values.
Reliable broadcast	A weaker version of reliable terminating broadcast(RTB), namely reliable broadcast, in which the termination condition is dropped, is solvable under crash failures.

(i) Terminating Reliable Broadcast (TRB) Problem

- A correct process always gets a message, even if sender crashes while sending (in which case the process gets a null message).
- **Validity:** If the sender of a broadcast message m is non-faulty, then all correct processes eventually deliver m .
- **Agreement:** If a correct process delivers a message m , then all correct processes deliver m .
- **Integrity:** Each correct process delivers at most one message. Further, if it delivers a message different from the null message, then the sender must have broadcast m .
- **Termination:** Every correct process eventually delivers some message.

Contd...

- The reduction from consensus to terminating reliable broadcast is as follows:
- A commander process broadcasts its input value using the terminating reliable broadcast. A process decides on a “0” or “1” depending on whether it receives “0” or “1” in the message from this process.
- However, if it receives the null message, it decides on a default value. As the broadcast is done using the terminating reliable broadcast, it can be seen that the conditions of the consensus problem are satisfied.
- But as consensus is not solvable, an algorithm to implement terminating reliable broadcast cannot exist.

(ii) Reliable Broadcast Problem

- **Reliable Broadcast** is RTB without terminating condition.
- RTB requires eventual delivery of messages, even if sender fails before sending. In this case, a null message needs to get sent. In RB, this condition is not there.
- RTB requires recognition of a failure, even if no msg is sent
- Crux: RTB is required to distinguish between a failed process and a slow process.
- RB is solvable under crash failures; $O(n^2)$ messages

Algorithm: Protocol for reliable broadcast

(1) Process P_0 initiates Reliable Broadcast:

(1a) **broadcast** message M to all processes.

(2) A process $P_i, 1 \leq i \leq n$, receives message M :

(2a) **if** M was not received earlier **then**

(2b) **broadcast** M to all processes;

(2c) deliver M to the application.

Applications of Agreement Algorithms

1) Fault-Tolerant Clock Synchronization

- Distributed Systems require physical clocks to be synchronized
- Physical clocks have a drift problem
- Agreement Protocols may help to reach a common clock value.

2) Atomic Commit in Distributed Database System (DDBS)

- DDBS sites must agree whether to commit or abort the transaction
- Agreement protocols may help to reach a consensus.

Conclusion

- **Consensus problems are fundamental aspects** of distributed computing because they **require inherently distributed processes** to reach agreement.
- This lecture first covers:
 - Different forms of the **consensus problem**,
 - Then gives an overview of what forms of consensus are solvable under different failure models and different assumptions on the synchrony/asynchrony.

Contd...

- Then we have covered agreement in the following categories:

(i) Synchronous message-passing systems with failures:

-Used Fault Models: fail-stop model and the Byzantine model.

(ii) Asynchronous message-passing systems with failures:

-Impossible to reach consensus in this model.

-Hence, several weaker versions of the consensus problem, i.e. terminating reliable broadcast, reliable broadcast are considered.

- In upcoming lecture, we will discuss about the **'Checkpointing and Rollback Recovery'**.

Lecture: 11

Checkpointing and Rollback Recovery



Rajiv Misra

Dept. of Computer Science &
Engineering

Indian Institute of Technology Patna

rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have discussed about different forms of the '**consensus problem**' then gives an overview of what forms of consensus are solvable under different failure models and different assumptions on the synchrony/asynchrony.
- Also covered agreement in the category of:
(i) Synchronous message-passing systems with failures
and (ii) Asynchronous message-passing systems with failures.

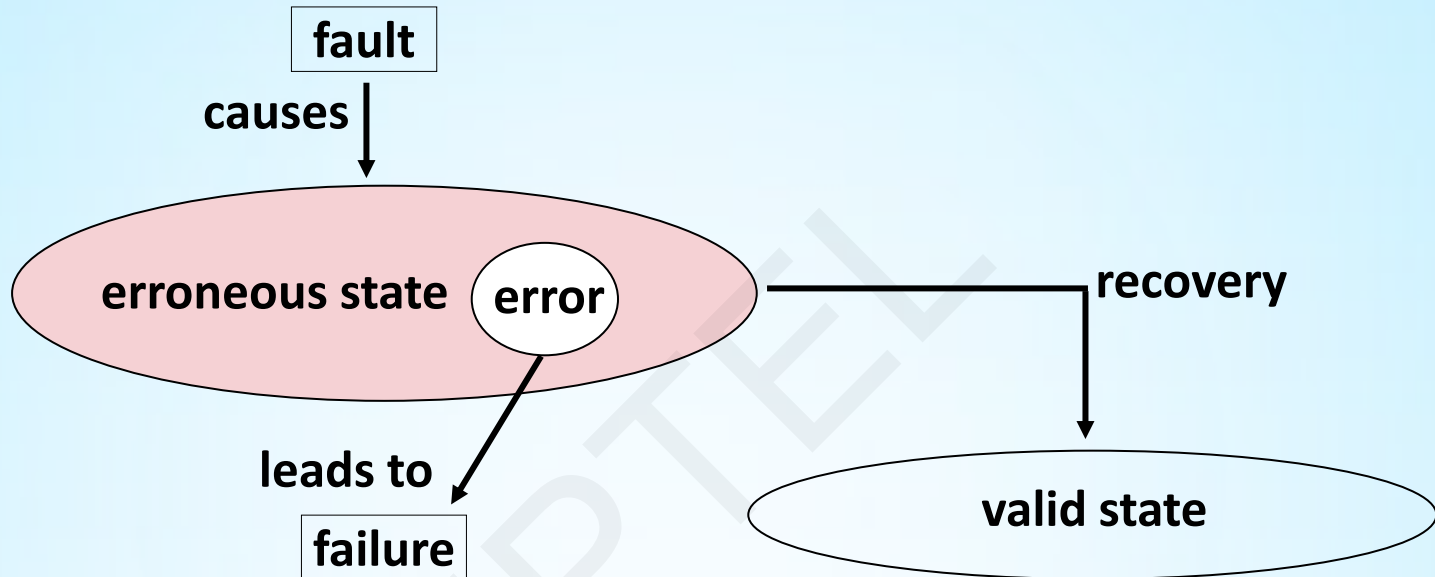
Preface

Content of this Lecture:

- In this lecture, we will discuss about basic fundamentals and underlying concepts of '**Checkpointing and Rollback Recovery**' and
- Also discuss different **Rollback Recovery Schemes** i.e.
 - (i) Checkpoint based and
 - (ii) Log based

Introduction

- **Fault Tolerance**



- An error is a manifestation of a fault that can lead to a failure.

- **Failure Recovery:**

- Backward recovery- Restore system state to a previous error-free state
 - operation-based (do-undo-redo logs)
 - state-based (checkpointing/logging)
- Forward recovery- Repair the erroneous part of the system state

Introduction

- **Rollback recovery protocols**
 - **Restore the system** back to a **consistent state** after a failure
 - **Achieve fault tolerance** by periodically saving the state of a process during the failure-free execution
 - Treats a distributed system application as a collection of processes that communicate over a network
- **Checkpoints**
 - **The saved states** of a process
- **Why is rollback recovery of distributed systems complicated?**
 - Messages induce inter-process dependencies during failure-free operation
- **Rollback propagation**
 - The dependencies may force some of the processes that did not fail to roll back
 - This phenomenon is called “**domino effect**”

Contd...

- If each process takes its checkpoints independently, then the system **can not avoid the domino effect**
 - this scheme is called **independent or uncoordinated checkpointing**
- Techniques **that avoid domino effect**
 - **Coordinated checkpointing rollback recovery**
 - processes coordinate their checkpoints to form a system-wide consistent state
 - **Communication-induced** checkpointing rollback recovery
 - **forces each process to take checkpoints** based on information piggybacked on the application
 - **Log-based rollback recovery**
 - combines checkpointing with logging of non-deterministic events relies on piecewise deterministic (PWD) assumption

Preliminaries

A local checkpoint

- All processes save their **local states** at certain instants of time
- A local check point is a **snapshot of the state** of the process at a given instance

Assumptions:

- A process stores all local checkpoints on the stable storage
- A process is able to roll back to any of its existing local checkpoints
- $C_{i,k}$
 - The kth local checkpoint at process P_i
- $C_{i,o}$
 - A process P_i takes a checkpoint $C_{i,o}$ before it starts execution

Consistent states

- **A global state of a distributed system**

- a collection of the individual states of all participating processes and the states of the communication channels

- **Consistent global state**

- a global state that may occur during a failure-free execution of distribution of distributed computation

- if a process's state reflects a message receipt, then the state of the corresponding sender must reflect the sending of the message

- **A global checkpoint**

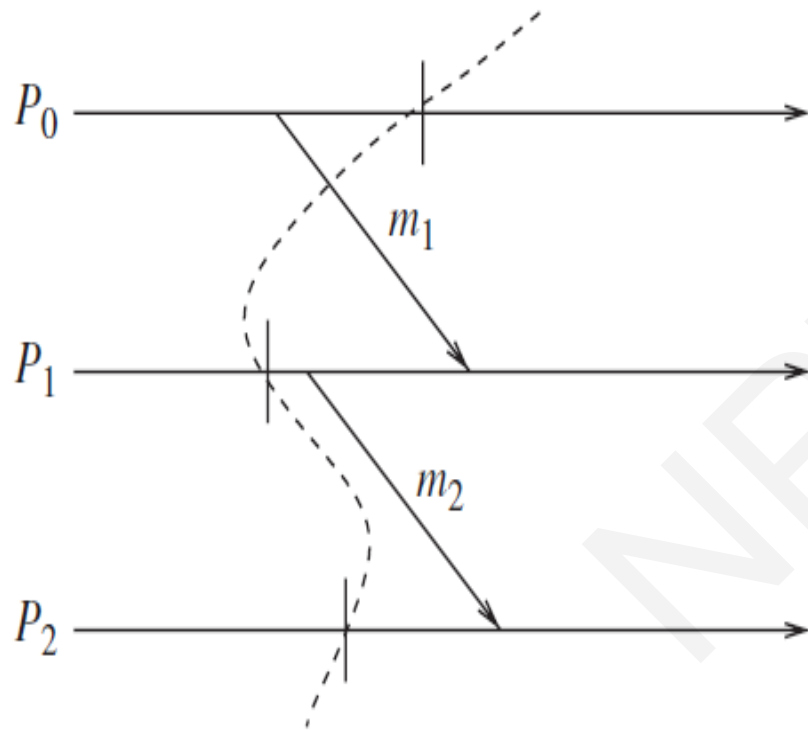
- a set of local checkpoints, one from each process

- **A consistent global checkpoint**

- a global checkpoint such that no message is sent by a process after taking its local point that is received by another process before taking its checkpoint

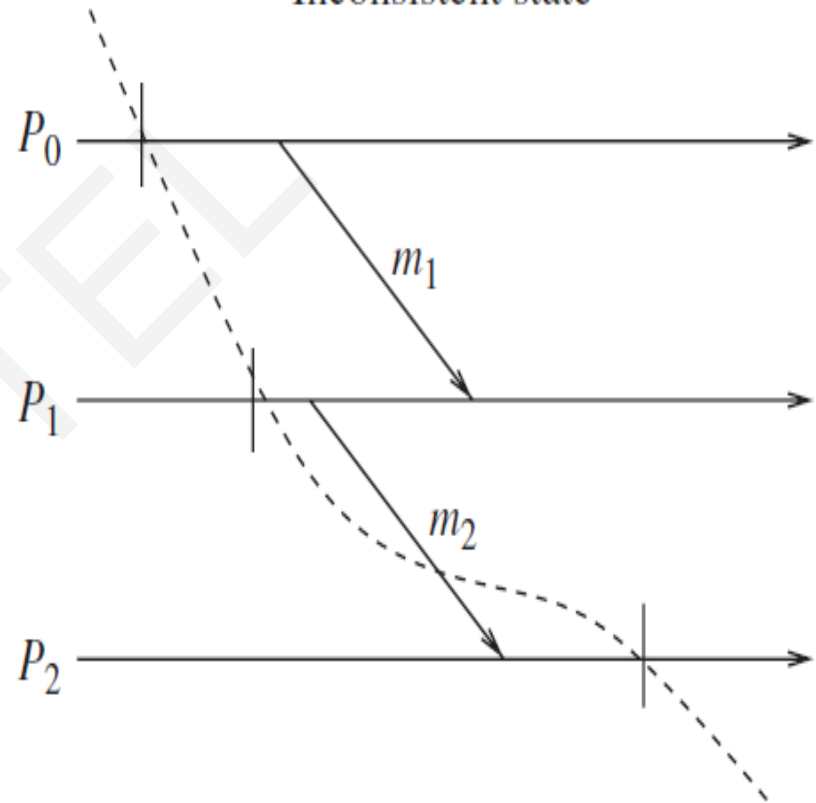
Consistent Examples

Consistent state



(a)

Inconsistent state



(b)

Interactions with outside world

- A distributed system often interacts with the outside world to receive input data or deliver the outcome of a computation
- **Outside World Process (OWP)**
 - a special process that interacts with the rest of the system through message passing
- **A common approach**
 - save each input message on the stable storage before allowing the application program to process it
- **Symbol “| |”**
 - An interaction with the outside world to deliver the outcome of a computation

Messages

- **In-transit message**

- messages that have been sent but not yet received

- **Lost messages**

- messages whose 'send' is done but 'receive' is undone due to rollback

- **Delayed messages**

- messages whose 'receive' is not recorded because the receiving process was either down or the message arrived after rollback

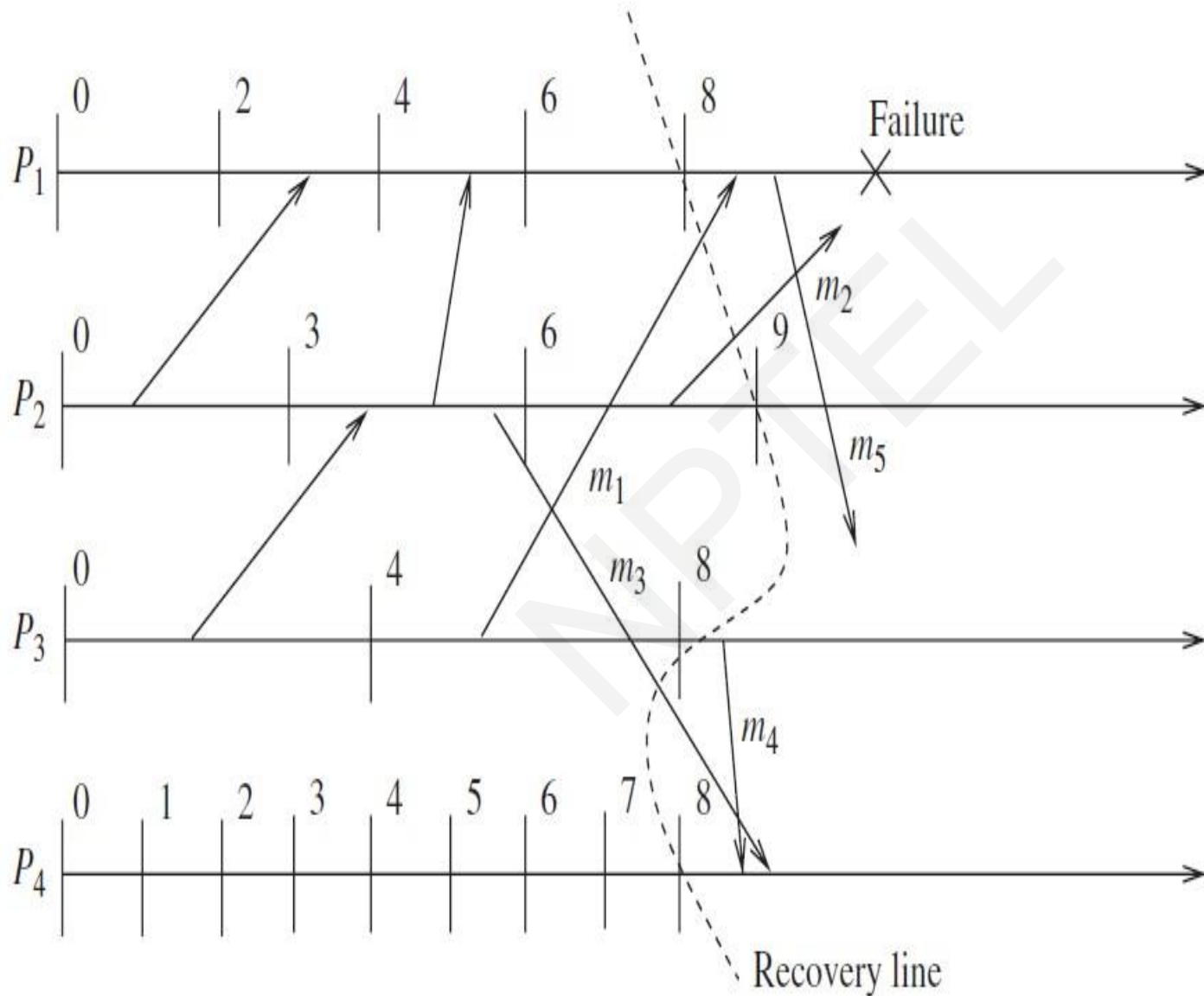
- **Orphan messages**

- messages with 'receive' recorded but message 'send' not recorded
 - do not arise if processes roll back to a consistent global state

- **Duplicate messages**

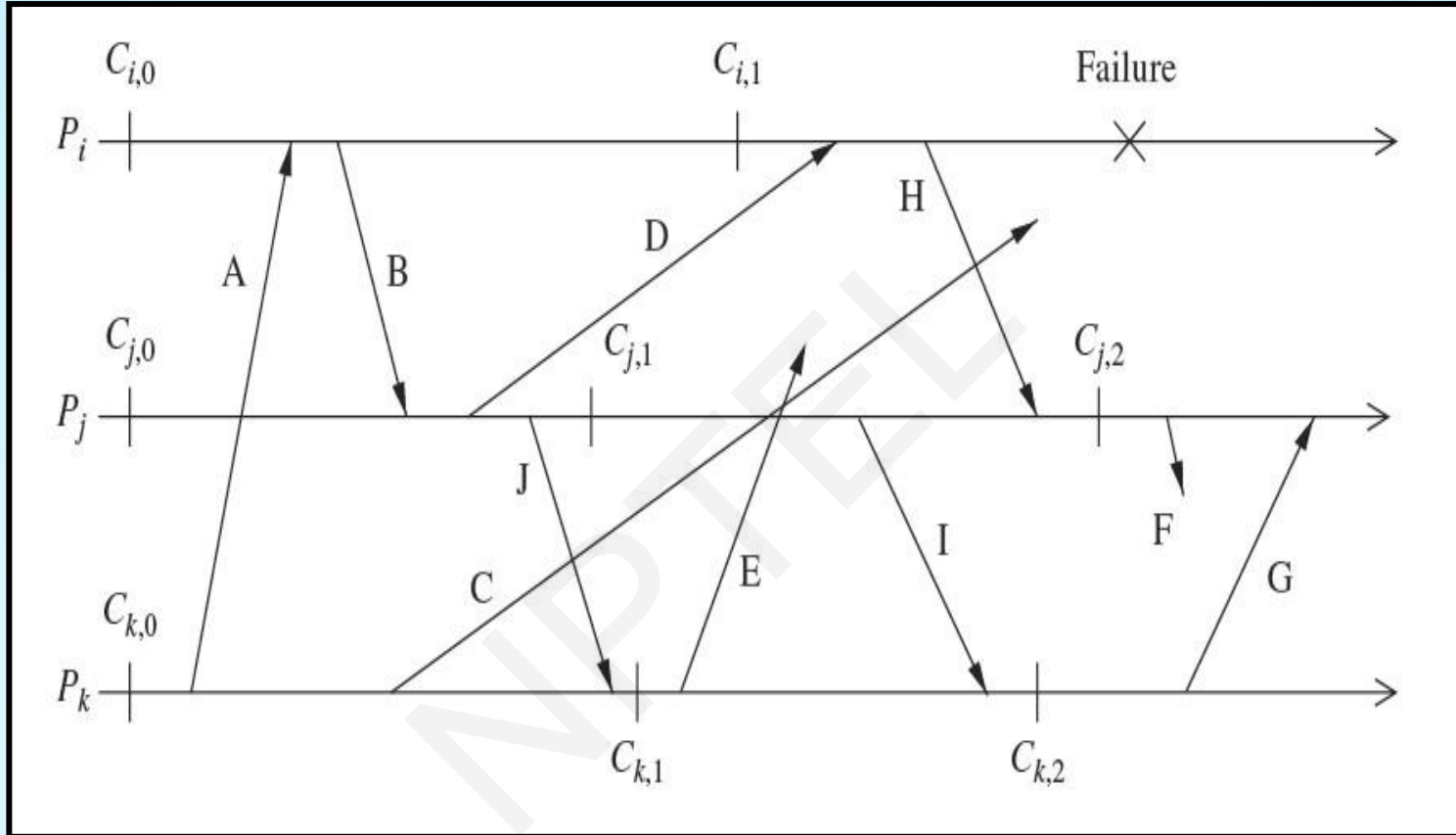
- arise due to message logging and replaying during process recovery

Messages: Example



- **In-transit**
 - m_1, m_2
- **Lost**
 - m_1
- **Delayed**
 - m_1, m_5
- **Orphan**
 - none
- **Duplicated**
 - m_4, m_5

Issues in failure recovery



- **Checkpoints :** $\{C_{i,0}, C_{i,1}\}$, $\{C_{j,0}, C_{j,1}, C_{j,2}\}$, and $\{C_{k,0}, C_{k,1}, C_{k,2}\}$
- **Messages :** A - J
- **The restored global consistent state :** $\{C_{i,1}, C_{j,1}, C_{k,1}\}$

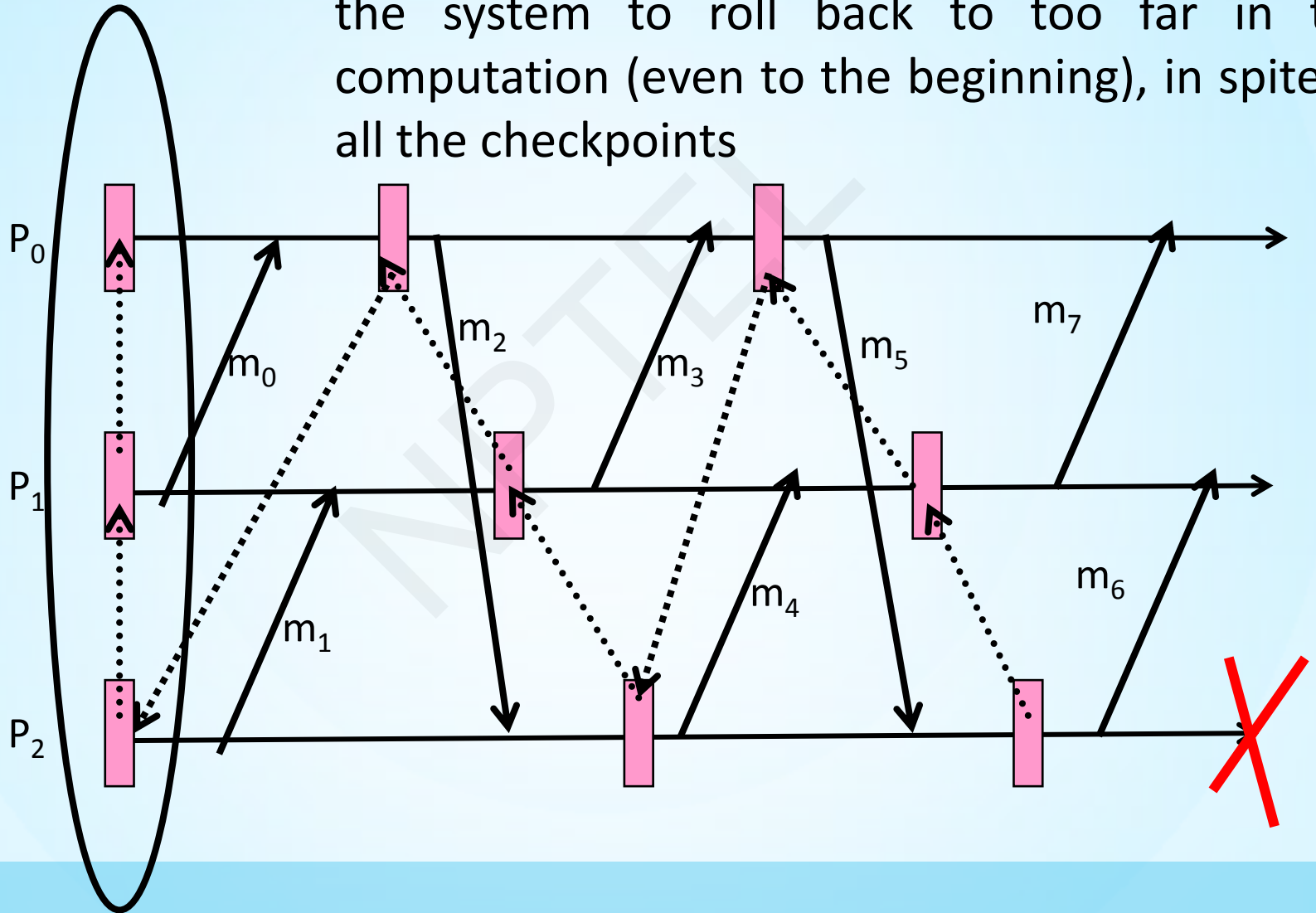
Issues in failure recovery

- The rollback of process P_i to checkpoint $C_{i,1}$ created an orphan message H
- Orphan message I is created due to the roll back of process P_j to checkpoint $C_{j,1}$
- Messages C, D, E, and F are potentially problematic
 - Message C: a delayed message
 - Message D: a lost message since the send event for D is recorded in the restored state for P_j , but the receive event has been undone at process P_i .
 - Lost messages can be handled by having processes keep a message log of all the sent messages
 - Messages E, F: delayed orphan messages. After resuming execution from their checkpoints, processes will generate both of these messages

Domino effect: example

Recovery Line

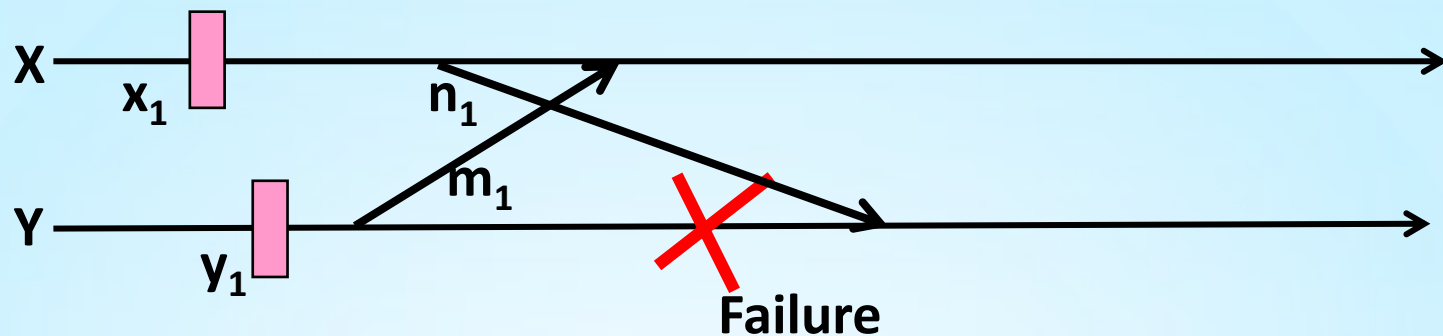
Domino Effect: Cascaded rollback which causes the system to roll back to too far in the computation (even to the beginning), in spite of all the checkpoints.



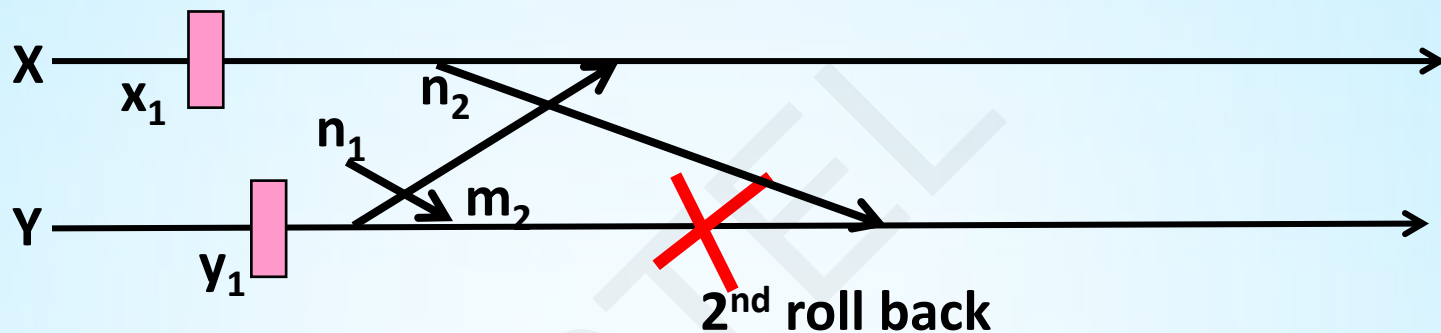
Problem of Livelock

- **Livelock:** case where a single failure can cause an infinite number of rollbacks.
- **The Livelock problem** may arise when a process rolls back to its checkpoint after a failure and requests all the other affected processes also to roll back.
- In such a situation if the roll back mechanism has no synchronization, it may lead to the livelock problem as described in the example 11.2.

Case-I



Case-II



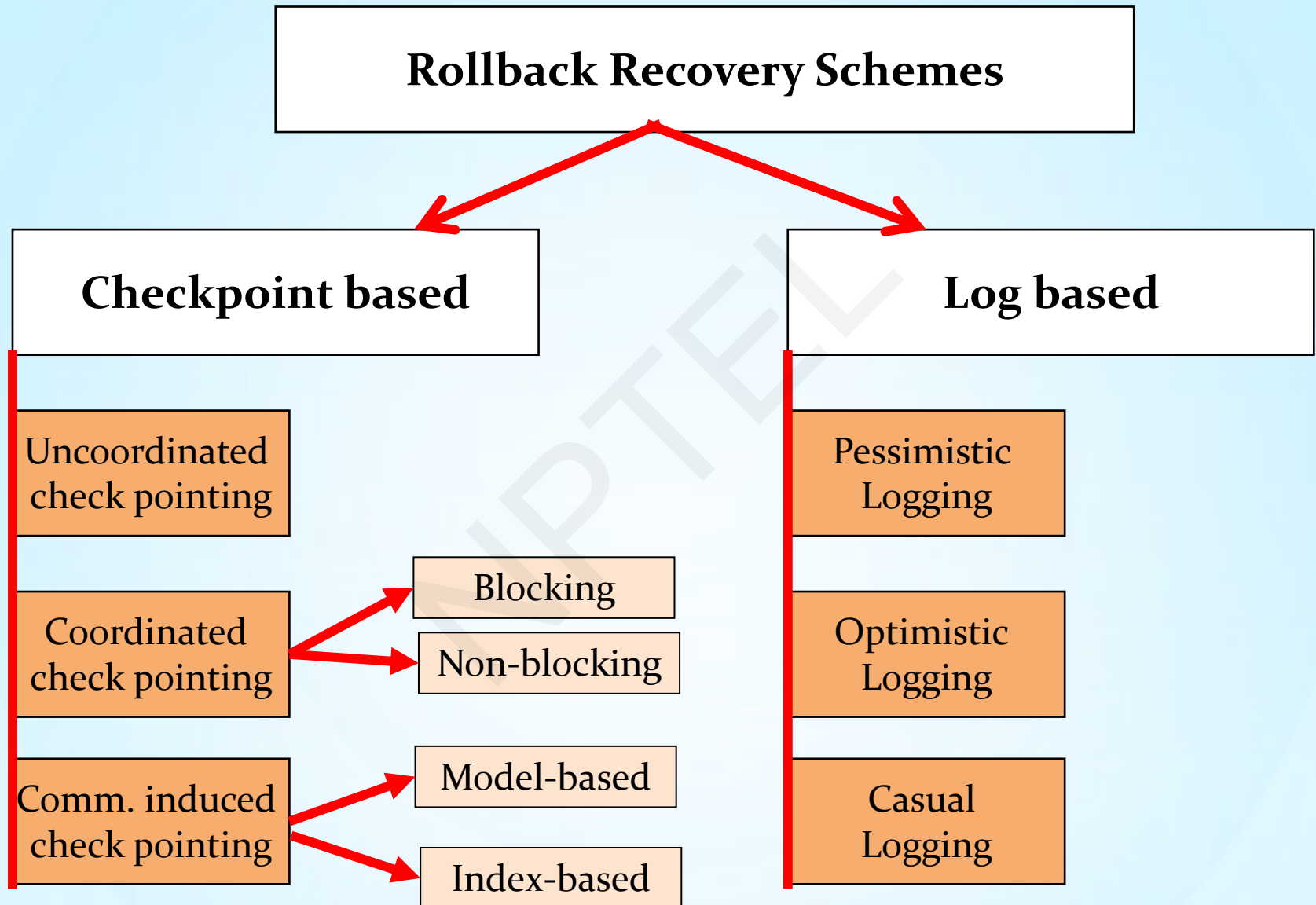
Case-I

- Process Y fails before receiving message 'n1' sent by X
- Y rolled back to y_1 , no record of sending message 'm1', causing X to rollback to x_1

Case-II

- When Y restarts, sends out 'm2' and receives 'n1' (delayed)
- Y has to roll back again, since there is no record of 'n1' being sent
- This cause X to be rolled back again, since it has received 'm2' and there is no record of sending 'm2' in Y
- The above sequence can repeat indefinitely

Different Rollback Recovery Schemes



Checkpoint Based Recovery Schemes

Checkpoint Based Recovery: Overview

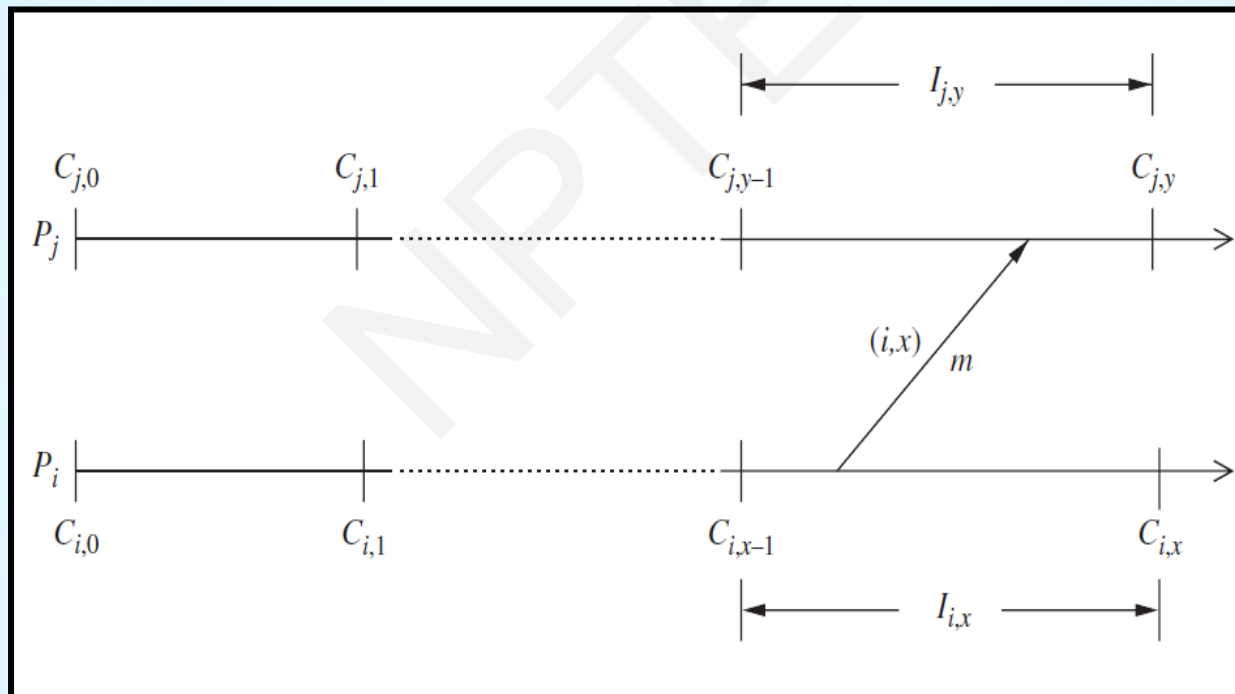
- 1. Uncoordinated checkpointing:** Each process takes its checkpoints independently
- 2. Coordinated checkpointing:** Process coordinate their checkpoints in order to save a system-wide consistent state. This consistent set of checkpoints can be used to bound the rollback
- 3. Communication-induced checkpointing:** It forces each process to take checkpoints based on information piggybacked on the application messages it receives from other processes.

1. Uncoordinated Checkpointing

- Each process has autonomy in deciding when to take checkpoints
- **Advantages**
 - The **lower runtime overhead** during normal execution
- **Disadvantages**
 - **Domino effect** during a recovery
 - **Recovery from a failure is slow** because processes need to iterate to find a consistent set of checkpoints
 - Each process maintains **multiple checkpoints** and periodically invoke a garbage collection algorithm
 - Not suitable for application with frequent output commits
- The processes record the dependencies among their checkpoints caused by message exchange during failure-free operation

Example: Direct dependency tracking technique

- Assume each process P_i starts its execution with an initial checkpoint $C_{i,0}$
- $I_{i,x}$: checkpoint interval, interval between $C_{i,x-1}$ and $C_{i,x}$
- When P_j receives a message m during $I_{j,y}$, it records the dependency from $I_{i,x}$ to $I_{j,y}$, which is later saved onto stable storage when P_j takes $C_{j,y}$



2. Coordinated Checkpointing

- **Blocking Checkpointing**

- After a process takes a local checkpoint, to prevent orphan messages, it remains blocked until the entire checkpointing activity is complete

- **Disadvantages**

- The computation is blocked during the checkpointing

- **Non-blocking Checkpointing**

- The processes need not stop their execution while taking checkpoints

- A fundamental problem in coordinated checkpointing is to prevent a process from receiving application messages that could make the checkpoint inconsistent.

Example

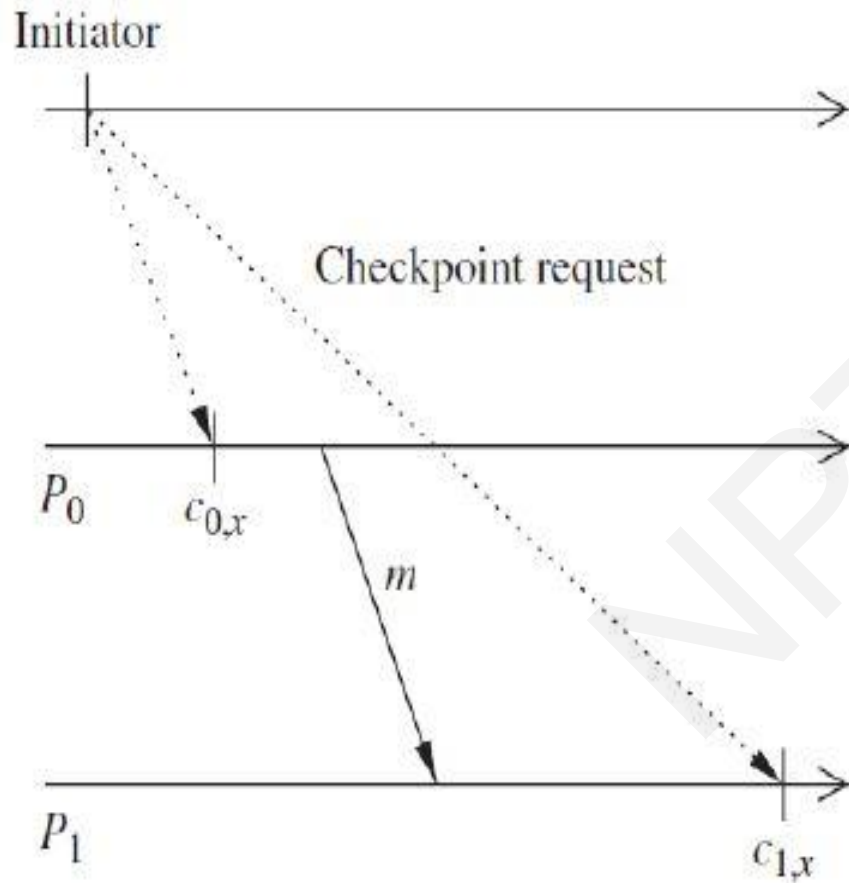
Example (a) : checkpoint inconsistency

- message m is sent by P_0 after receiving a checkpoint request from the checkpoint coordinator
- Assume m reaches P_1 before the checkpoint request
- This situation results in an inconsistent checkpoint since checkpoint
- $C_{1,x}$ shows the receipt of message m from P_0 , while checkpoint $C_{0,x}$ does not show m being sent from P_0

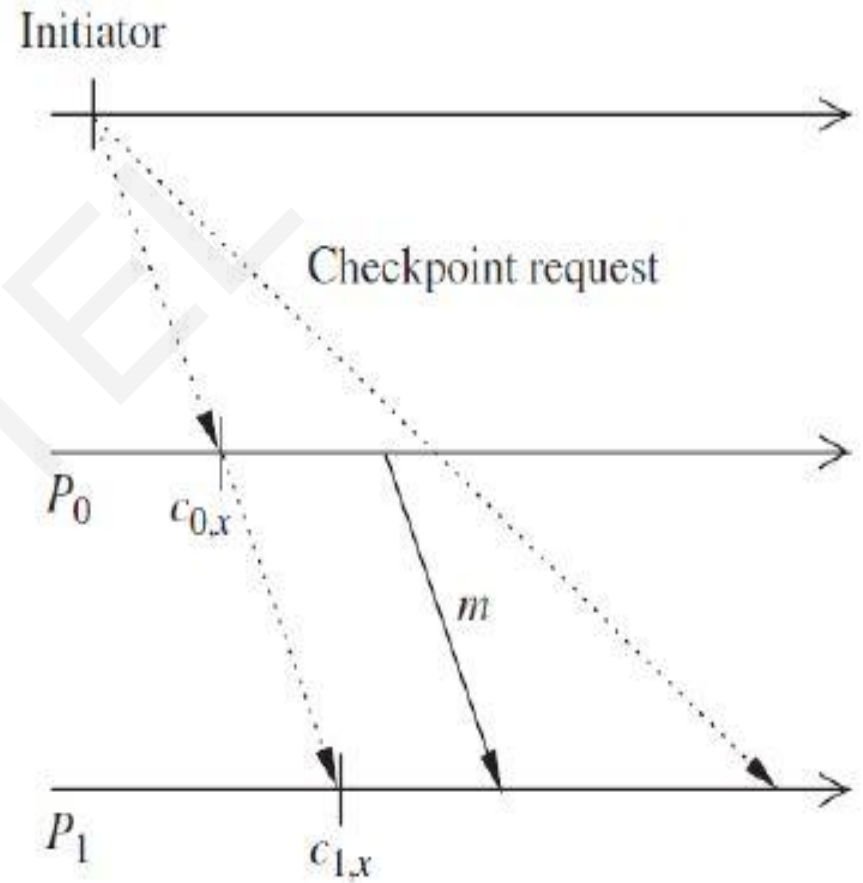
Example (b) : a solution with FIFO channels

- If channels are FIFO, this problem can be avoided by preceding the first post-checkpoint message on each channel by a checkpoint request, forcing each process to take a checkpoint before receiving the first post-checkpoint message.

Contd...



(a)



(b)

3. Communication-induced Checkpointing

- **Two types of checkpoints**

- autonomous and forced checkpoints

- Communication-induced checkpointing piggybacks protocol- related information on each application message
- The receiver of each application message uses the piggybacked information to determine if it has to take a forced checkpoint to advance the global recovery line
- The forced checkpoint must be taken before the application may process the contents of the message
- In contrast with coordinated checkpointing, no special coordination messages are exchanged

Contd...

- **Two types of communication-induced checkpointing**
 - model-based checkpointing and
 - index-based checkpointing.

In '***model-based checkpointing***', the system **maintains checkpoints and communication structures** that prevent the domino effect or achieve some even stronger properties.

In '***index-based checkpointing***', the system **uses an indexing scheme** for the local and forced checkpoints, such that the checkpoints of the same index at all processes form a consistent state.

Log-based Rollback Recovery Schemes

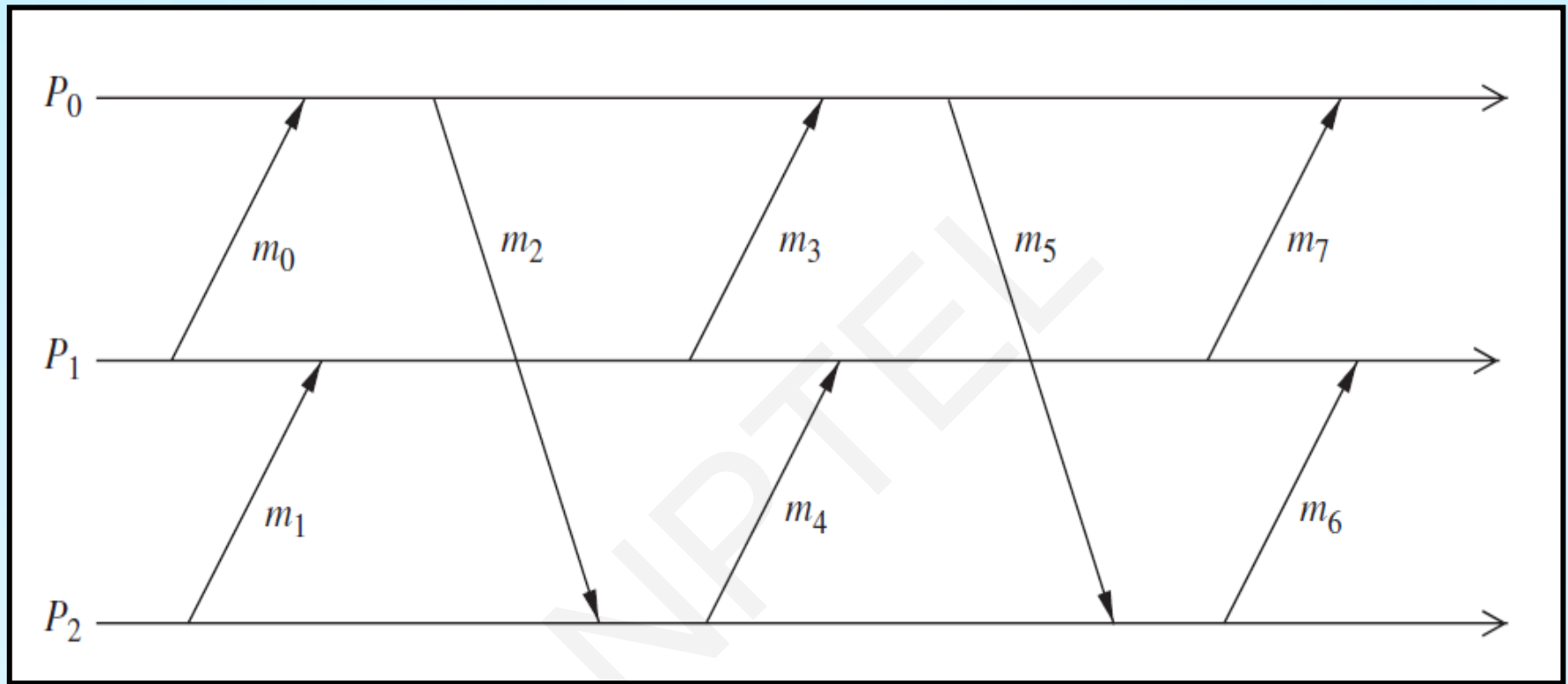
Log-based Rollback Recovery: Overview

- **It combines checkpointing with logging of nondeterministic events.**
- It relies on the **piecewise deterministic (PWD) assumption**, which postulates that all nondeterministic events that a process executes can be identified and that the information necessary to replay each event during recovery can be logged in the event's determinant (all info. necessary to replay the event).
- By logging and replaying the nondeterministic events in their exact original order, a process can deterministically recreate its pre-failure state even if this state has not been checkpointed.
- Log-based rollback recovery is in general attractive for applications that frequently interact with the outside world which consists of input and output logged to stable storage.

Contd...

- A log-based rollback recovery makes **use of deterministic and nondeterministic events in a computation.**
- **Deterministic and Non-deterministic events**
 - Non-deterministic events can be the receipt of a message from another process or an event internal to the process
 - a message send event is *not* a non-deterministic event.
 - the execution of process P_0 is a sequence of four deterministic intervals
 - Log-based rollback recovery assumes that all non-deterministic events can be identified and their corresponding determinants can be logged into the stable storage
 - During failure-free operation, each process logs the determinants of all non-deterministic events that it observes onto the stable storage

Deterministic and Non-deterministic events: Example



The execution of process P_0 is a sequence of four deterministic intervals. The first one starts with the creation of the process, while the remaining three start with the receipt of messages m_0 , m_3 , and m_7 , respectively. Send event of message m_2 is uniquely determined by the initial state of P_0 and by the receipt of message m_0 , and is therefore not a non-deterministic event.

No-orphans consistency condition

- Let e be a non-deterministic event that occurs at process p
- **$Depend(e)$**
 - the set of processes that are affected by a non-deterministic event e . This set consists of p , and any process whose state depends on the event e according to **Lamport's *happened before* relation**
- **$Log(e)$**
 - the set of processes that have logged a copy of e 's determinant in their volatile memory
- **$Stable(e)$**
 - a predicate that is true if e 's determinant is logged on the stable storage
- **$always\text{-}no\text{-}orphans\ condition$**
 - $\forall(e) : \neg Stable(e) \Rightarrow Depend(e) \subseteq Log(e)$

Log-based recovery schemes

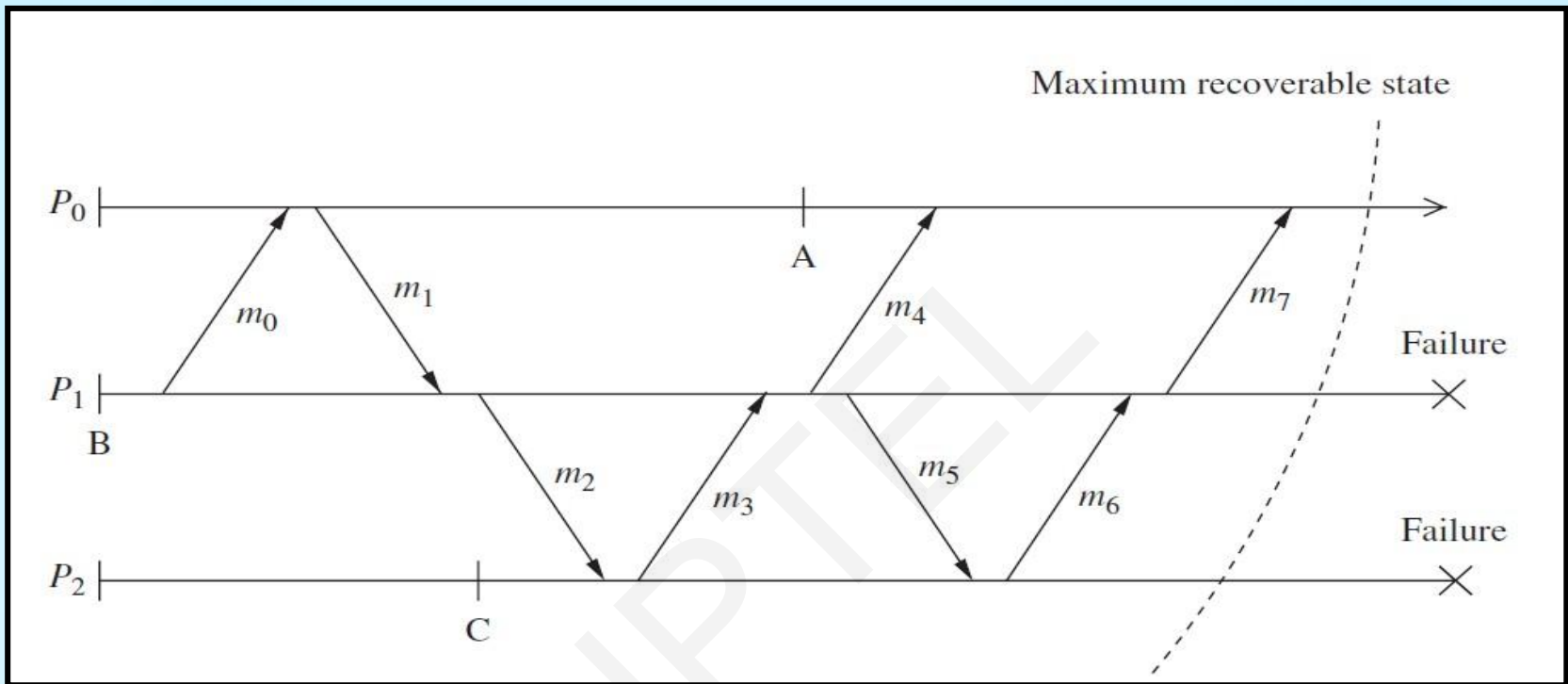
- Schemes differ in the way the determinants are logged into the stable storage.

- 1. Pessimistic Logging:** The application has to block waiting for the determinant of each nondeterministic event to be stored on stable storage before the effects of that event can be seen by other processes or the outside world. It simplifies recovery but hurts the failure-free performance.
- 2. Optimistic Logging:** The application does not block, and the determinants are spooled to stable storage asynchronously. It reduces failure free overhead, but complicates recovery.
- 3. Casual Logging:** Low failure free overhead and simpler recovery are combined by striking a balance between optimistic and pessimistic logging.

1. Pessimistic Logging

- Pessimistic logging protocols assume that a failure can occur after any non-deterministic event in the computation
- However, in reality failures are rare
- ***synchronous logging***
 - $\forall e: \neg \text{Stable}(e) \Rightarrow |\text{Depend}(e)| = 0$
 - if an event has not been logged on the stable storage, then no process can depend on it.
 - stronger than the always-no-orphans condition

Pessimistic Logging: Example

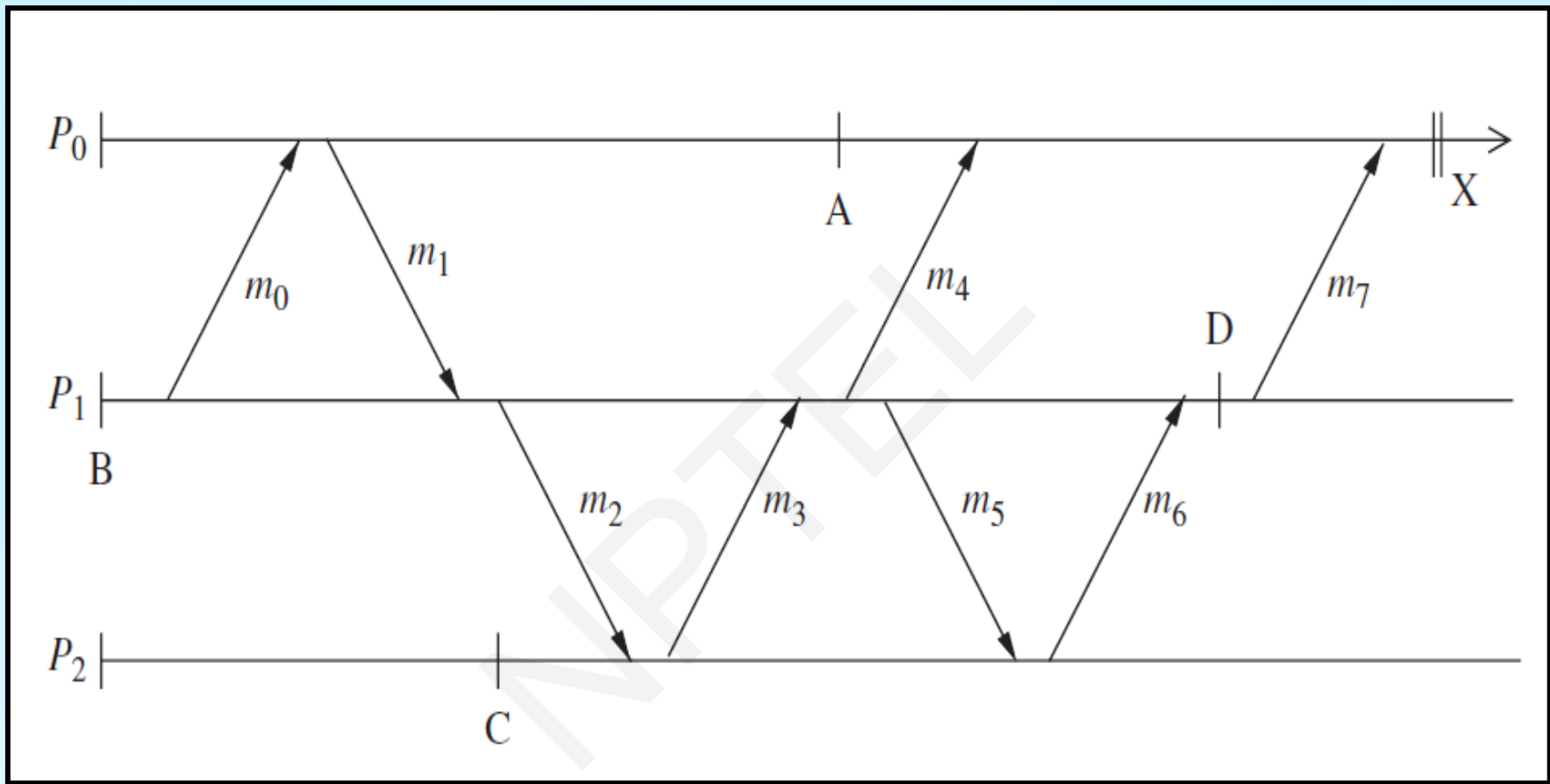


- Suppose processes P_1 and P_2 fail as shown, restart from checkpoints B and C , and roll forward using their determinant logs to deliver again the same sequence of messages as in the pre-failure execution
- Once the recovery is complete, both processes will be consistent with the state of P_0 that includes the receipt of message m_7 from P_1

2. Optimistic Logging

- Processes log determinants asynchronously to the stable storage
- Optimistically assume that logging will be complete before a failure occurs
- Do not implement the *always-no-orphans* condition
- To perform rollbacks correctly, optimistic logging protocols track causal dependencies during failure free execution
- Optimistic logging protocols require a non-trivial garbage collection scheme
- Pessimistic protocols need only keep the most recent checkpoint of each process, whereas optimistic protocols may need to keep multiple checkpoints for each process

Optimistic Logging: Example

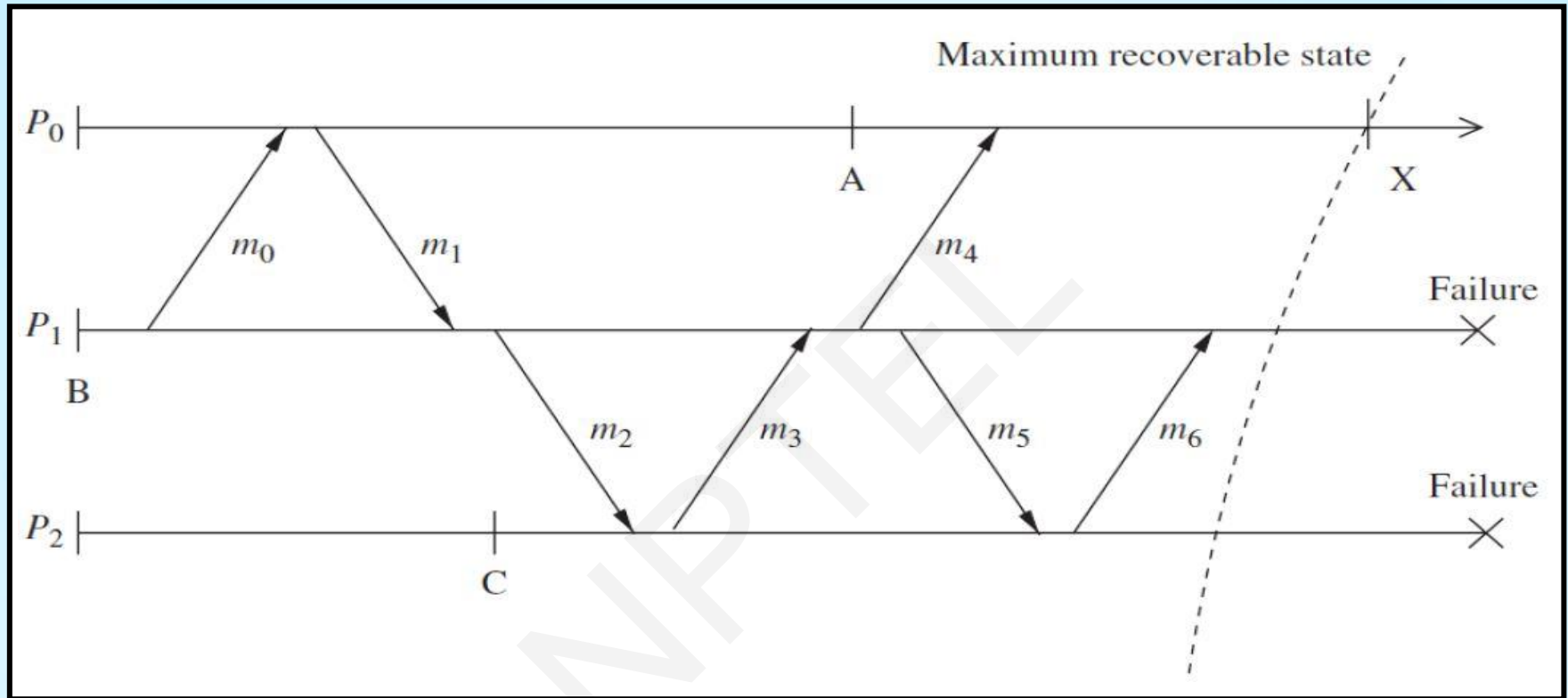


Consider the example shown in figure. Suppose process P_2 fails before the determinant for m_5 is logged to the stable storage. Process P_1 then becomes an orphan process and must roll back to undo the effects of receiving the orphan message m_6 . The rollback of P_1 further forces P_0 to roll back to undo the effects of receiving message m_7 .

3. Causal Logging

- Combines the advantages of both pessimistic and optimistic logging at the expense of a more complex recovery protocol
- Like optimistic logging, it does not require synchronous access to the stable storage except during output commit
- Like pessimistic logging, it allows each process to commit output independently and never creates orphans, thus isolating processes from the effects of failures at other processes
- Make sure that the always-no-orphans property holds
- Each process maintains information about all the events that have causally affected its state

Causal Logging: Example



Messages m_5 and m_6 are likely to be lost on the failures of P_1 and P_2 at the indicated instants. Process P_0 at state X will have logged the determinants of the nondeterministic events that causally precede its state according to Lamport's happened-before relation. These events consist of the delivery of messages m_0 , m_1 , m_2 , m_3 , and m_4 . The determinant of each of these non-deterministic events is either logged on the stable storage or is available in the volatile log of process P_0 .

Checkpointing and Recovery Algorithms

Koo-Toueg Coordinated Checkpointing Algorithm

- **Koo and Toueg (1987)** proposed a coordinated checkpointing and recovery technique that takes a consistent set of checkpointing and **avoids 'domino effect' and 'livelock problems'** during the recovery
- It Includes 2 parts:
 - (i) The checkpointing algorithm and
 - (ii) The recovery algorithm

Contd...

(i) The Checkpointing Algorithm

- **Assumptions:** FIFO channel, end-to-end protocols, communication failures do not partition the network, single process initiation, no process fails during the execution of the algorithm
- Two kinds of checkpoints: permanent and tentative
 - **Permanent checkpoint:** local checkpoint, part of a consistent global checkpoint
 - **Tentative checkpoint:** temporary checkpoint, become permanent checkpoint when the algorithm terminates successfully

Contd...

Checkpointing Algorithm:

–2 phases

1. The initiating process takes a tentative checkpoint and requests all other processes to take tentative checkpoints. Every process can not send messages after taking tentative checkpoint. All processes will finally have the single same decision: do or discard
2. All processes will receive the final decision from initiating process and act accordingly

–Correctness: for 2 reasons

- Either all or none of the processes take permanent checkpoint
- No process sends message after taking permanent checkpoint

–Optimization: maybe not all of the processes need to take checkpoints (if not change since the last checkpoint)

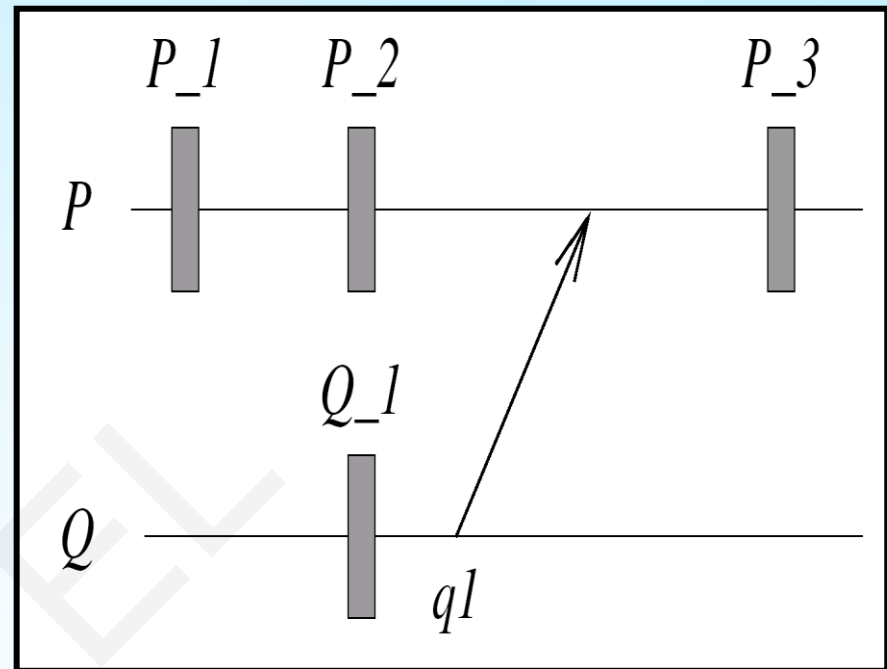
Contd...

(ii) The Rollback Recovery Algorithm:

- Restore the system state to a consistent state after a failure with assumptions: single initiator, checkpoint and rollback recovery algorithms are not invoked concurrently
- **2 phases**
 1. The initiating process send a message to all other processes and ask for the preferences – restarting to the previous checkpoints. All need to agree about either do or not.
 2. The initiating process send the final decision to all processes, all the processes act accordingly after receiving the final decision.

Example

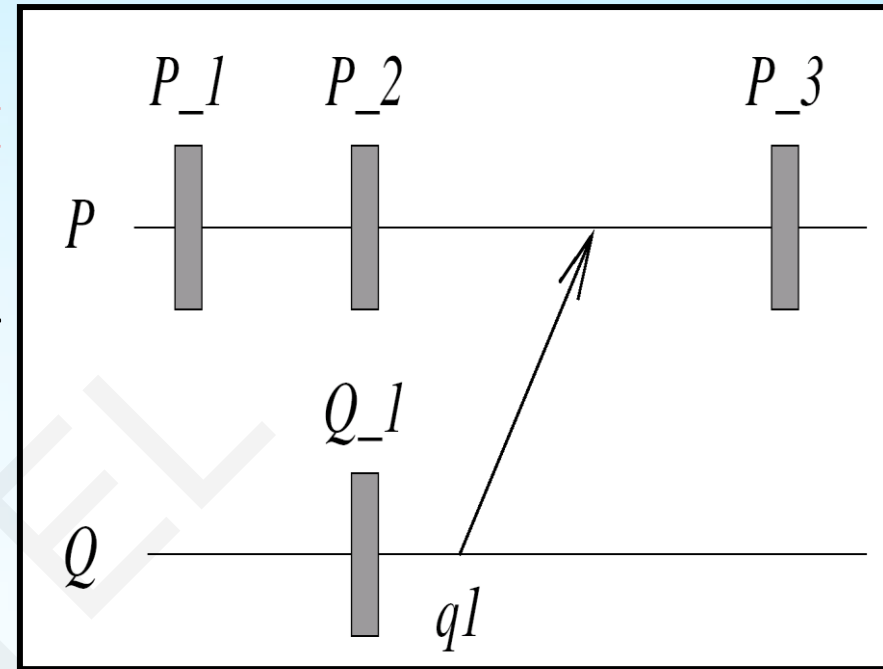
Suppose **P** wants to establish a checkpoint at **P_3**. This will record that **q1** was received from **Q** - to prevent q1 from being orphaned, Q must checkpoint as well.



- Thus, establishing a checkpoint at **P_3** by **P** forces **Q** to take a checkpoint to record that **q1** was sent
- An algorithm for such coordinated checkpointing has two types of checkpoints - tentative and permanent
- **P** first records its current state in a tentative checkpoint, then sends a message to all other processes from whom it has received a message since taking its last checkpoint
- Call the set of such processes **Π**

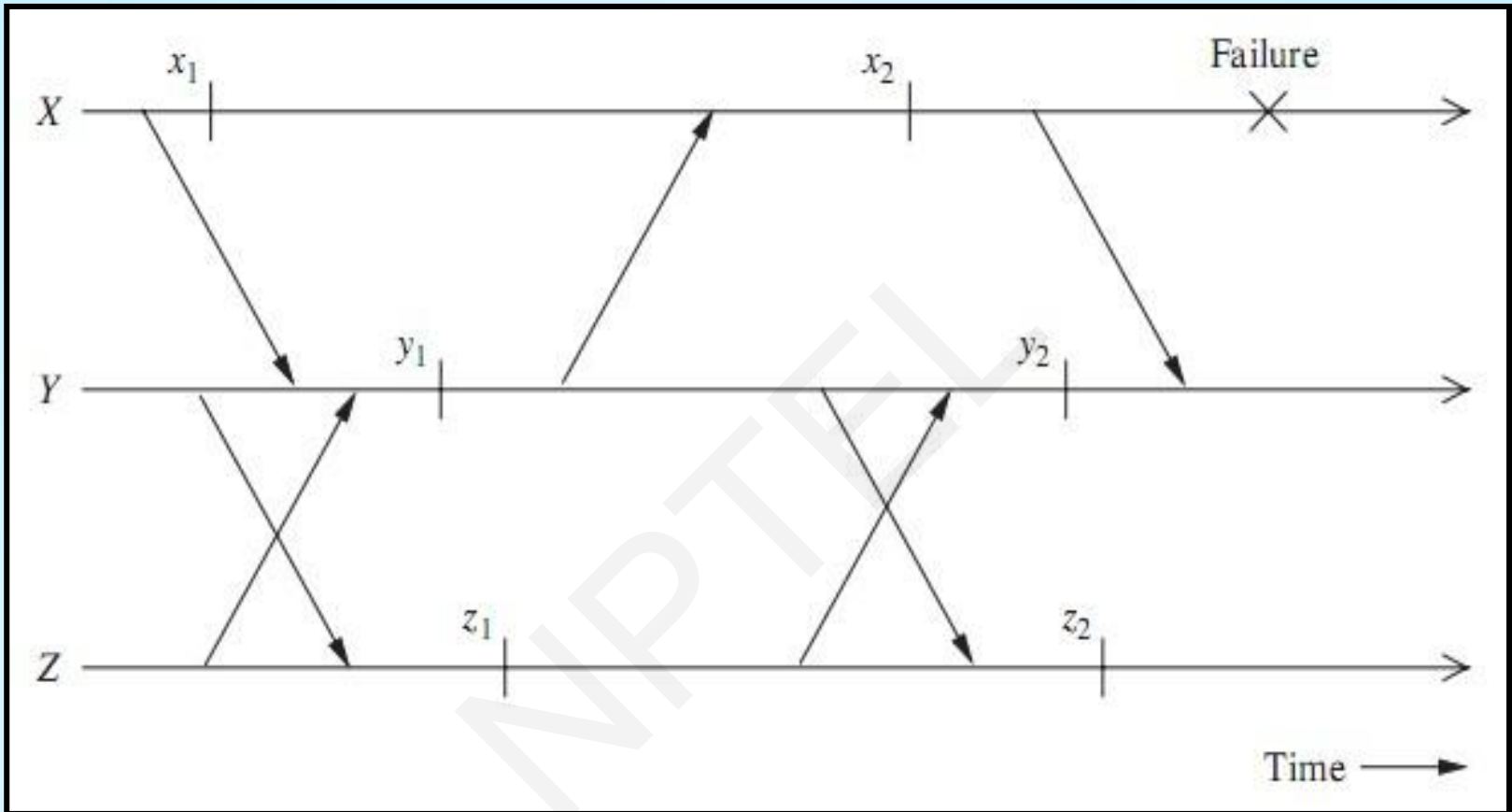
Contd...

The message tells each process in Π (e.g., Q), the last message, m_{qp} , that P has received from it before the tentative checkpoint was taken. If m_{qp} was not recorded in a checkpoint by Q : to prevent m_{qp} from being orphaned, Q is asked to take a tentative checkpoint to record sending m_{qp} .



- If all processes in Π , that need to, confirm taking a checkpoint as requested, then all tentative checkpoints can be converted to permanent
- If some members of Π , are unable to checkpoint as requested, P and all members of Π abandon the tentative checkpoints, and none are made permanent
- This may set off a chain reaction of checkpoints
- Each member of Π can potentially spawn a set of checkpoints among processes in its corresponding set

Contd...



- **Correctness:** resume from a consistent state
- **Optimization:** may not to recover all, since some of the processes did not change anything

Few Other Algorithms for Checkpointing and Recovery

Algorithm	Basic Idea
Juang–Venkatesan (1991) algorithm for asynchronous checkpointing and recovery	Since the algorithm is based on asynchronous checkpointing, the main issue in the recovery is to find a consistent set of checkpoints to which the system can be restored. The recovery algorithm achieves this by making each processor keep track of both the number of messages it has sent to other processors as well as the number of messages it has received from other processors.
Manivannan–Singhal (1996) quasi-synchronous checkpointing algorithm	The Manivannan–Singhal quasi-synchronous checkpointing algorithm improves the performance by eliminating useless checkpoints. The algorithm is based on communication-induced checkpointing, where each process takes basic checkpoints asynchronously and independently, and in addition, to prevent useless checkpoints, processes take forced checkpoints upon the reception of messages with a control variable.
Peterson–Kearns (1993) algorithm based on vector time	The Peterson–Kearns checkpointing and recovery protocol is based on the optimistic rollback. Vector time is used to capture causality to identify events and messages that become orphans when a failed process rolls back.
Helary–Mostefaoui–Netzer–Raynal (2000, 1997) communication-induced protocol	The Helary–Mostefaoui–Netzer–Raynal communication-induced checkpointing protocol prevents useless checkpoints and does it efficiently. To prevent useless checkpoints, some coordination is required in taking local checkpoints.

Conclusion

- **Rollback recovery achieves fault tolerance** by periodically saving the state of a process during the failure-free execution, and restarting from a saved state on a failure to reduce the amount of lost computation.
- There are three basic approaches for checkpointing and failure recovery: (i) uncoordinated, (ii) coordinated, and (iii) communication induced checkpointing and for log based: (i) Pessimistic, (ii) Optimistic and (iii) Casual Logging
- Over the last two decades, checkpointing and failure recovery has been a very active area of research and several checkpointing and failure recovery algorithms have been proposed. In this lecture, we described '**Koo-Toueg Coordinated Checkpointing Algorithm**' and given an overview of other algorithms.
- In upcoming lecture, we will discuss about '**Deadlock Detection in Distributed Systems**'